Project Report

on

“CivicSense: Containerised Deployment of a Civic Complaint Intelligence Platform using Docker”

Submitted by

Mohammad Iqbal Patan

UID: 24MCC20081

MCA (CC & DevOps) · Semester 3 · Section 102/A

Under the guidance of

Sofia

Subject: Containerization With Docker | Subject Code: 25CAH-734

Class Group: 25CAH-734:P::GP-A

in partial fulfilment for the award of the degree of

MASTER OF COMPUTER APPLICATIONS
CLOUD COMPUTING & DEVOPS

University Institute of Computing (UIC)

Chandigarh University

October 2026

TABLE OF CONTENTS

Certificate **6**
Declaration **7**
Acknowledgement **8**
Abstract **9**
1. Introduction **10**
1.1 Background 10
1.2 Problem Statement 10
1.3 Why Containers 11
1.4 Proposed Solution 11
1.5 Scope of the Project 12
1.6 Organisation of the Report 12
2. Background and Project Objectives **13**
2.1 Virtual Machines and Containers 13
2.2 Docker Architecture 13
2.3 Docker Compose 14
2.4 Docker Swarm 14
2.5 The Application Being Containerised 14
2.6 Project Objectives 15
3. System Requirements **16**
3.1 Hardware Requirements 16
3.2 Software Requirements 16
3.3 Functional Requirements 17
3.4 Non-Functional Requirements 17
4. Tools and Technologies Used **18**
4.1 Docker Engine and BuildKit 18
4.2 Docker Compose and Docker Swarm 19
4.3 nginx and gunicorn 19
4.4 GitHub Actions and Kubernetes 19
5. System Design **20**
5.1 Container Architecture 20
5.2 Network and Volume Design 20
5.3 Start-up Sequence 21
5.4 Image Design: a Two-Stage Build 21
5.5 Data Model and Data Flow 22
5.6 Security Design 23
6. Implementation Steps **24**
6.1 Step 1: Externalise the Configuration 24
6.2 Step 2: Add a Health Endpoint 24
6.3 Step 3: Write the Dockerfile and .dockerignore 24
6.4 Step 4: Write the Entrypoint Script 24
6.5 Step 5: Describe the Stack with Docker Compose 24

6.6 Step 6: Configure nginx as Reverse Proxy....25
6.7 Step 7: Verify the Stack....25
6.8 Step 8: Continuous Integration....25
6.9 Step 9: Deploy on Docker Swarm....25
6.10 Step 10: Kubernetes Manifests....26
7. Implementation and Code Explanation....27
7.1 Project Structure....27
7.2 Twelve-Factor Configuration....27
7.3 The Health Endpoint....27
7.4 The Dockerfile....28
7.5 The Entrypoint Script....29
7.6 The Compose File....30
7.7 The nginx Configuration....31
7.8 The Swarm Stack File....32
7.9 The CI Workflow....33
7.10 The Kubernetes Manifests....34
8. Testing and Results....36
8.1 Testing Strategy....36
8.2 Build Results....36
8.3 Stack Bring-up and Health....37
8.4 Automated Tests inside the Container....40
8.5 Self-Healing with a Restart Policy....41
8.6 Docker Swarm: Replicas and Load Balancing....42
8.7 Scaling....43
8.8 Self-Healing in the Swarm....44
8.9 Rolling Update and Rollback....44
8.10 Resource Usage....45
8.11 Summary of Results....45
8.12 Defects Found and Fixed....46
9. Screenshots....47
10. Conclusion....52
11. Future Scope....53
References....54
Appendix A: Running and Demonstrating the Project....55

LIST OF FIGURES

S. No.	Figure	Page
1	Figure 5.1 – Container architecture of CivicSense under Docker Compose	20
2	Figure 5.2 – Start-up sequence of the stack: health-gated dependencies and the entrypoint	21
3	Figure 5.3 – Two-stage build: only the installed packages move to the runtime image	22
4	Figure 5.4 – Entity-relationship diagram of CivicSense	22
5	Figure 5.5 – Level-1 data flow diagram	23
6	Figure 6.1 – CI pipeline: no green tests, no image; every image is smoke-tested	25
7	Figure 6.2 – CivicSense deployed as a Docker Swarm stack on one node	26
8	Figure 8.1 – Image sizes (runtime 435 MB vs builder 698 MB) and the layers of the image	37
9	Figure 8.2 – docker compose ps: database and web healthy, only nginx published	38
10	Figure 8.3 – Container log: wait for MySQL, migrate, seed, then gunicorn with three workers	38
11	Figure 8.4 – Health-check history, restart policy and the non-root user	39
12	Figure 8.5 – Service-name DNS on the bridge network, named volumes, resource usage and data in MySQL	40
13	Figure 8.6 – Least privilege refuses the test database; with the admin account all 39 tests pass	41
14	Figure 8.7 – Crash simulation under Compose: exit 137, automatic restart, healthy after 11.3 s	41
15	Figure 8.8 – Swarm initialised, stack deployed, services converged (nginx healed itself once)	42
16	Figure 8.9 – One-off seeding task, service VIP and replica addresses, 90 requests split 30/30/30	43
17	Figure 8.10 – Scaling the web service from 3 to 5 replicas and back to 3	43
18	Figure 8.11 – Swarm self-healing: replica replaced, 522 requests OK and 0 failed	44
19	Figure 8.12 – Zero-downtime rolling update (840 OK, 0 failed) and rollback (622 OK, 0 failed)	45
20	Figure 9.1 – Portal home page served by the containers through nginx (localhost:8088), live statistics read from MySQL	47
21	Figure 9.2 – City complaints page served from the Docker stack	48
22	Figure 9.3 – Filing a complaint: category, deciding words and priority shown while the citizen types	48
23	Figure 9.4 – Duplicate warning: a similar complaint nearby is found before submission	49
24	Figure 9.5 – Authority dashboard: KPIs, SLA breaches and ward hot-spots	49
25	Figure 9.6 – Work queue ordered by the priority score with the SLA due date	50
26	Figure 9.7 – Duplicate clusters merged by the engine	50
27	Figure 9.8 – Department-wise SLA report	51
28	Figure 9.9 – Django administration of the containerised application	51

LIST OF TABLES

S. No.	Table	Page
1	Table 2.1 – Virtual machines compared with containers	13
2	Table 3.1 – Hardware requirements	16
3	Table 3.2 – Software requirements	16
4	Table 3.3 – Functional requirements of the containerised system	17
5	Table 4.1 – Tools and technologies used	18
6	Table 5.1 – Services of the Compose stack	20
7	Table 5.2 – Security measures in the container design	23
8	Table 7.1 – Dockerfile instructions explained	28
9	Table 8.1 – Build results	37
10	Table 8.2 – Measured resource usage (docker stats)	45
11	Table 8.3 – Summary of measured results	45
12	Table 8.4 – Defects found and fixed	46
13	Table A.1 – Demonstration accounts (seeded data)	55

CERTIFICATE

This is to certify that the project report entitled **"CivicSense: Containerised Deployment of a Civic Complaint Intelligence Platform using Docker"** submitted by **Mohammad Iqbal Patan** (UID: 24MCC20081), MCA (Cloud Computing & DevOps), Semester III, Section 102/A (class group 25CAH-734:P::GP-A), in partial fulfilment of the requirements for the award of the degree of Master of Computer Applications (Cloud Computing & DevOps) at Chandigarh University, is a record of bonafide work carried out by the student under my supervision and guidance as part of the course **Containerization With Docker (Subject Code: 25CAH-734)** during the academic session 2026–2027.

The project involved containerising a real Django and MySQL application with a multi-stage Dockerfile, orchestrating it with Docker Compose and Docker Swarm, verifying its health, self-healing, scaling and zero-downtime updates, and automating its build with a continuous integration pipeline. It demonstrates the practical application of containerisation and DevOps practices to a real-world problem.

To the best of my knowledge, the work embodied in this report has not been submitted, in part or in full, to any other university or institution for the award of any degree or diploma.

Sofia
Project Guide
University Institute of Computing

Head of Department
University Institute of Computing
Chandigarh University

Date: _____

Place: Chandigarh University, Mohali, Punjab

DECLARATION

I, **Mohammad Iqbal Patan**, student of Master of Computer Applications (Cloud Computing & DevOps), Semester III, University Institute of Computing, Chandigarh University, hereby declare that the project report entitled **"CivicSense: Containerised Deployment of a Civic Complaint Intelligence Platform using Docker"**, submitted for the course Containerization With Docker (25CAH-734), is an authentic record of my own work carried out under the guidance of Sofia.

All the code, configuration, measurements and screenshots presented in this report were produced while building, running and testing the project. Wherever the work of others has been used, it has been duly acknowledged in the text and listed in the references.

This report has not been submitted to any other university or institution for the award of any degree or diploma.

Date: _____
Place: Mohali, Punjab

Mohammad Iqbal Patan
UID: 24MCC20081

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to all those who contributed to the successful completion of this project titled **“CivicSense: Containerised Deployment of a Civic Complaint Intelligence Platform using Docker”**.

First and foremost, I would like to thank **Sofia**, my project guide, for her valuable guidance, continuous support and encouragement throughout the course of this project. Her suggestions and feedback played a key role in shaping both the system and this report.

I extend my heartfelt thanks to the faculty members of the University Institute of Computing, Chandigarh University, for providing the facilities, the curriculum in containerisation, cloud computing and DevOps, and a stimulating environment in which to carry out this work.

I am also grateful to the open-source communities behind Docker, Django, MySQL, nginx and gunicorn. Their freely available software and documentation made it possible to build and operate a production-style platform on a single laptop.

Lastly, I would like to thank my family, friends and classmates for their motivation, moral support and constructive feedback during the development of this project.

Mohammad Iqbal Patan
UID: 24MCC20081

ABSTRACT

Applications that run perfectly on a developer's laptop often fail on a server because the runtime, the libraries and the configuration differ. Containerisation removes this gap by packaging an application with everything it needs into an image that runs identically on any Docker host. This project applies Docker to a complete, real application rather than to a single demonstration container.

The application is **CivicSense**, a civic complaint intelligence platform built with Python, Django and MySQL, whose natural-language engine routes complaints to departments, merges repeat reports and scores their priority. It was packaged with a **two-stage Dockerfile** that compiles the MySQL driver in a discarded builder stage and ships a **435 MB** image (the builder stage is 698 MB) running as a non-root user with a built-in health check. An entrypoint script waits for the database, migrates and collects static files before starting gunicorn. **Docker Compose** brings up MySQL 8.4, the application and an nginx reverse proxy with one command, using a private network, health-gated start-up and named volumes. The same image was then deployed as a **Docker Swarm** stack with an overlay network, three load-balanced replicas, resource limits, rolling updates and rollback, and a GitHub Actions pipeline and Kubernetes manifests were written for it.

All 39 automated tests passed inside the container against MySQL 8.4. A crashed container was restarted in 1.1 s and was healthy in 11.3 s. In the swarm, 90 requests were shared exactly 30/30/30 between three replicas; the service scaled to five replicas in 44 s; a killed replica was replaced in 15.9 s with 522 requests served and none failed; and a rolling update and a rollback completed with 840 and 622 requests respectively and no failures. Testing the real deployment also revealed defects invisible during development – data re-seeded on every restart and logins rejected by a CSRF origin check – which were diagnosed from container logs and fixed in configuration.

The source code, Docker files and this report are available at https://github.com/iPatan7/civicsense-docker.

Keywords: Docker, Containerisation, Dockerfile, Multi-stage Build, Docker Compose, Docker Swarm, Self-healing, Rolling Update, Load Balancing, Django, MySQL, nginx, CI/CD, DevOps.

CHAPTER 1: INTRODUCTION

1.1 Background

Modern software is no longer delivered as a single program copied onto a server. It is a set of cooperating processes – an application server, a database, a reverse proxy, background jobs – each with its own runtime, libraries and configuration. Keeping all of these identical on a developer's laptop, on a test machine and on a production server has always been the hardest part of deployment, and it is the reason behind the familiar complaint *"it works on my machine"*.

Containerisation solves this by packaging an application together with everything it needs – the language runtime, system libraries, Python packages and start-up scripts – into a single, versioned **image**. The image runs as an isolated **container** on any host that has a container engine, using the host's Linux kernel but its own file system, process table and network stack. Docker, released in 2013, made this technique practical for everyday developers through a simple command line, a text-based build recipe (the *Dockerfile*) and a public registry of ready-made images (Docker Hub).

Docker is now the standard unit of delivery in cloud computing and DevOps. Docker Compose describes a whole multi-container system in one file, Docker Swarm turns a group of Docker hosts into a cluster that keeps a requested number of replicas running, and the same images are run unchanged by Kubernetes and by every major cloud provider. Learning to containerise a real application – not a single "hello world" container – is therefore a core skill for a cloud and DevOps engineer.

1.2 Problem Statement

The application chosen for this project is **CivicSense**, a civic complaint intelligence platform built in Experiment 5 of Technical Training-II. Citizens file complaints about potholes, garbage, water supply or street lights in plain language; a natural-language engine routes each complaint to the right department, detects repeat reports of the same problem and scores its priority; officers work through a ranked queue on an authority dashboard. The application is written in Python with Django and stores its data in MySQL.

Before containerisation, running CivicSense outside the developer's laptop suffered from five concrete problems:

- **Environment drift** – the Python version, the MySQL client library (`mysqlclient` needs a C compiler and MySQL header files to install) and the static files differed between machines, so code that ran in development failed on the server.

- **Manual, ordered releases** – every release required waiting for the database, running migrations, collecting static files and restarting the server in exactly the right order; a forgotten step brought the portal down.
- **Configuration in the code** – secret keys and database passwords written in `settings.py` meant a different build per environment and a security leak the day the repository was shared.
- **No resilience or scaling** – a single process on a single machine could not be updated without downtime, did not restart itself after a crash, and could not absorb a sudden rise in complaints after heavy rain.
- **No gate before deployment** – nothing ran the tests automatically against a production-like database, so broken code could reach the server.

The problem addressed by this project is therefore: **how can CivicSense be packaged and operated with Docker so that the same image runs identically everywhere, the whole system starts with one command, failures are healed automatically and the application can be scaled and updated without downtime?**

1.3 Why Containers

Virtual machines also isolate applications, but each virtual machine carries a full guest operating system, takes minutes to boot and consumes gigabytes of memory. A container shares the host kernel and starts in about a second. In this project a complete CivicSense web container – Python 3.12, Django, gunicorn and the application – used about 120–150 MiB of memory, and a crashed container was restarted by Docker within one second. This efficiency is what makes it practical to run several replicas of the same service on one machine and to replace them one at a time during an update.

1.4 Proposed Solution

The project containerises CivicSense end to end and operates it with the Docker toolchain:

- a **two-stage Dockerfile** that compiles the Python dependencies in a throw-away builder stage and ships a smaller runtime image that runs as an unprivileged user and carries its own **HEALTHCHECK**;
- an **entrypoint script** that turns a release into one ordered, repeatable procedure – wait for MySQL, migrate, collect static files, then hand over to gunicorn as PID 1;
- a **Docker Compose** file that brings up MySQL 8.4, the Django application under gunicorn and an nginx reverse proxy with one command, connected by a private network and backed by named volumes;

- a **Docker Swarm stack** that runs the same image as a replicated service with an overlay network, a routing mesh, resource limits, self-healing, scaling, rolling updates and rollback;
- a **GitHub Actions** pipeline that tests the code against a real MySQL service and then builds and smoke-tests the image, plus **Kubernetes manifests** for running the same image in a cluster.

1.5 Scope of the Project

The scope covers the containerisation, local orchestration (Compose), clustered orchestration on a single-node swarm, continuous integration and the documentation of a Kubernetes deployment. All Docker results in this report – image sizes, start-up times, test results, health checks, load balancing, self-healing, scaling and rolling-update figures – were produced on the author's machine on 30 October 2026 with Docker Engine 29.9.0 and Docker Compose v5.6.0. The design of the natural-language engine itself belongs to Experiment 5 and is described here only as far as it is needed to understand what is being deployed.

The complete source code, all Docker files and this report are published in a public repository: **https://github.com/iPatan7/civicsense-docker**.

1.6 Organisation of the Report

Chapter 2 explains the background concepts and lists the objectives. Chapter 3 states the hardware and software requirements, and Chapter 4 the tools used. Chapter 5 presents the container architecture, the image design and the data model. Chapter 6 lists the implementation steps, and Chapter 7 explains every Docker file line by line. Chapter 8 reports the testing and the measured results, including the defects that were found and fixed. Chapter 9 collects the screenshots, Chapter 10 and 11 give the conclusion and future scope, and Appendix A is a step-by-step guide for running and demonstrating the project.

CHAPTER 2: BACKGROUND AND PROJECT OBJECTIVES

2.1 Virtual Machines and Containers

A virtual machine virtualises hardware: a hypervisor presents virtual CPUs, memory and disks, and a complete guest operating system boots on top. A container virtualises the operating system instead: it is an ordinary Linux process that the kernel isolates using **namespaces** (its own view of process IDs, network interfaces, mount points, host name and users) and limits using **control groups** (CPU, memory and I/O quotas). Table 2.1 summarises the difference.

Table 2.1 – Virtual machines compared with containers

Aspect	Virtual machine	Container (Docker)
Isolation unit	Full guest OS on a hypervisor	Process isolated by namespaces and cgroups
Start-up time	Tens of seconds to minutes	About one second (measured: restart in 1.1 s)
Size	Gigabytes (whole OS image)	Megabytes; CivicSense image 435 MB incl. Python
Memory overhead	Guest kernel and services	Only the application (web ≈ 120–150 MiB)
Portability	Hypervisor-specific image formats	OCI image runs on any Docker/Kubernetes host
Density	A few per host	Dozens per host

2.2 Docker Architecture

Docker follows a client–server model. The **Docker CLI** sends requests to the **Docker daemon** (`dockerd`), which builds images, pulls them from registries and creates containers through `containerd` and the low-level runtime `runc`. The main objects are:

- **Image** – a read-only template made of stacked file-system **layers**. Each Dockerfile instruction that changes files creates a layer; layers are cached and shared between images, so a rebuild only re-executes the steps after the first change.
- **Container** – a running instance of an image with a thin writable layer on top. Containers are disposable: anything that must survive them is stored in a volume.
- **Volume** – storage managed by Docker that outlives containers (here the MySQL data, the static files and the uploaded complaint photographs).
- **Network** – a virtual network. On a user-defined bridge network Docker runs an embedded DNS server, so containers reach one another by service name (`db`, `web`) instead of by IP address.

- **Registry** – a store of images such as Docker Hub, from which `python:3.12-slim`, `mysql:8.4` and `nginx:1.27-alpine` were pulled.

2.3 Docker Compose

Docker Compose describes a multi-container application declaratively in a YAML file: the services, the image or build context of each, their environment variables, ports, volumes, health checks and start-up dependencies. `docker compose up` creates the network, volumes and containers in the right order, and `docker compose down` removes them again while keeping the named volumes. Compose is the natural tool for a developer machine, a test server or a single-host deployment.

2.4 Docker Swarm

Docker Swarm mode turns one or more Docker hosts into a cluster managed by the Docker engine itself. Instead of containers, the user declares **services** with a desired number of **replicas**; swarm **managers** keep the actual state equal to the desired state by starting, stopping and replacing **tasks**. Swarm provides:

- **Overlay networks** that span all nodes, with a **virtual IP (VIP)** per service and built-in load balancing across its replicas;
- an **ingress routing mesh**, so a published port reaches the service from any node;
- **self-healing** – a task that crashes or fails its health check is replaced automatically;
- **scaling** with one command (`docker service scale`), **rolling updates** with configurable parallelism, delay and order, and automatic or manual **rollback**;
- **configs and secrets** distributed to the tasks that need them, and **resource limits and reservations** per task.

The same Compose file format (with a `deploy:` section) is used for a swarm **stack**, which made it straightforward to move CivicSense from Compose to Swarm.

2.5 The Application Being Containerised

CivicSense consists of three Django applications (`accounts`, `complaints`, `dashboard`) and a natural-language engine (`nlpengine`) written with only the Python standard library: a Multinomial Naive Bayes classifier that routes a complaint to one of ten services and six departments, TF-IDF and cosine similarity combined with GPS distance to merge repeat reports, a lexicon-based sentiment analyser, and an explainable 100-point priority score that also sets the service-level (SLA) deadline. The application keeps no state in the process –

sessions live in the database – which is what later allows several replicas to run side by side behind a load balancer.

2.6 Project Objectives

The objectives of the project are:

- To package CivicSense into a **small, secure, reproducible Docker image** using a multi-stage build, a non-root user, a `.dockerignore` file and an in-image health check.
- To move all configuration into **environment variables** (twelve-factor style), so that one image runs with SQLite on a laptop and with MySQL in production.
- To bring up the complete system – **MySQL 8.4, Django under gunicorn and nginx** – with a single `docker compose up`, with correct start-up ordering, private networking and persistent volumes.
- To verify the deployment by running the **39 automated tests inside the container** against MySQL, and by checking health, logs, networking and resource usage.
- To demonstrate **self-healing, scaling, load balancing, rolling updates and rollback** with Docker Swarm, and to measure whether users notice any of these operations.
- To automate the build with a **CI pipeline** that refuses to build an image from failing code and smoke-tests every image, and to document a **Kubernetes** deployment of the same image.
- To record honestly the **defects** that only appeared when the application was run the way it is deployed, and how they were fixed.

CHAPTER 3: SYSTEM REQUIREMENTS

This chapter lists the resources used to build and run the project. The "used in this project" column records the actual machine and versions on which every result in Chapter 8 was measured.

3.1 Hardware Requirements

Table 3.1 – Hardware requirements

Component	Minimum	Used in this project
Processor	64-bit dual core with virtualisation	Intel Core i7-10510U, 4 cores / 8 threads, 1.8 GHz
Memory (RAM)	4 GB	14.9 GiB available to Docker
Storage	5 GB free for images and volumes	NVMe SSD; images ≈ 1.6 GB (MySQL 1.12 GB, app 435 MB, nginx 74.5 MB)
Network	Internet to pull base images	Broadband; one pull timed out and was retried

3.2 Software Requirements

Table 3.2 – Software requirements

Software	Version	Purpose
Operating system	Ubuntu 26.04.1 LTS	Docker host (any Linux, or Windows/macOS with Docker Desktop)
Docker Engine	29.9.0	Builds images and runs containers; Swarm mode
Docker Compose	v5.6.0	Multi-container definition and orchestration on one host
Base image	python:3.12-slim (3.12.16)	Runtime of the application image
Database image	mysql:8.4 (8.4.13 LTS)	Production database; Django 6 requires MySQL 8.4+
Proxy image	nginx:1.27-alpine (1.27.5)	Reverse proxy, static and media files
Django	6.1.2	Web framework of CivicSense
gunicorn / mysqlclient	26.2.0 / 2.3.0	WSGI application server / MySQL driver
WhiteNoise	6.12.0	Compressed, hashed static files
GitHub Actions	ubuntu-latest runners	Continuous integration
Kubernetes (optional)	1.30	Target of the manifests in deploy/k8s
Browser, curl	Chrome 155, curl 8	Using and probing the portal

3.3 Functional Requirements

Table 3.3 – Functional requirements of the containerised system

ID	Requirement
FR1	The whole platform (database, application, proxy) must start with one command and stop with one command without losing data.
FR2	The application container must not start serving before the database accepts connections, and must apply migrations and collect static files by itself.
FR3	Only the reverse proxy may be reachable from outside; the application and database ports must stay on the private network (the database is published on 3307 only for inspection).
FR4	Every container must report its health so that Docker, Swarm, the CI pipeline and Kubernetes can act on it.
FR5	The same image must run on SQLite in development and on MySQL in deployment, configured only through environment variables.
FR6	The web tier must run as several replicas behind one address, survive the loss of a replica and be updatable one replica at a time.
FR7	Every push must run the tests against MySQL and build and smoke-test the image; no image may be built from failing code.

3.4 Non-Functional Requirements

- **Portability** – identical behaviour on any Docker host; no host-specific path or IP address in the configuration.
- **Security** – no secrets in the image or repository, a non-root process, no compiler in the shipped image, the smallest practical attack surface.
- **Availability** – no failed user requests during a replica crash, a scale operation, a rolling update or a rollback.
- **Performance** – fast rebuilds through layer caching; a warm start of the stack in well under a minute.
- **Maintainability** – every operational step written down as code (Dockerfile, entrypoint, Compose and stack files, CI workflow) rather than in someone's memory.

CHAPTER 4: TOOLS AND TECHNOLOGIES USED

Table 4.1 lists the tools and technologies used in the project and the role each one plays.

Table 4.1 – Tools and technologies used

Tool	Role	How it is used in CivicSense
Docker Engine	Container runtime	Builds the two-stage image with BuildKit, runs containers, enforces health checks and restart policies
Dockerfile	Image recipe	Builder stage compiles mysqlclient; runtime stage adds a non-root user, HEALTHCHECK, ENTRYPOINT and CMD
Docker Compose	Single-host orchestration	Three services (db, web, nginx), one bridge network, three named volumes, health-based start order
Docker Swarm	Cluster orchestration	Replicated services, overlay network, routing mesh, configs, scaling, self-healing, rolling update, rollback
Docker Hub	Image registry	Source of python:3.12-slim, mysql:8.4 and nginx:1.27-alpine
Python 3.12 / Django 6.1	Application	CivicSense web application, ORM, migrations, admin, test runner
gunicorn	WSGI server	Runs Django with three worker processes inside the web container
nginx	Reverse proxy	Single public entry point; serves /static and /media from shared volumes; forwards headers
MySQL 8.4	Database	Production database in its own container with a persistent volume and a health check
WhiteNoise	Static files	Compressed, cache-busted static files collected into the static volume
GitHub Actions	Continuous integration	Test job with a MySQL service container; image build and smoke test job
Kubernetes	Cloud orchestration	Deployment with probes and rolling update, Service, HPA, migration Job, ConfigMap, Secret
curl, docker stats, docker events	Verification	Health probes, load-balancing checks, resource and event monitoring

4.1 Docker Engine and BuildKit

BuildKit, Docker's build engine, executes independent stages in parallel. In this project the builder stage (installing the compiler and building the wheels) and the runtime stage (installing the MySQL client and creating the user) were built at the same time, and the whole image was built from scratch in **187 seconds**, including downloading the Python base image. Later rebuilds re-use every cached layer up to the first changed instruction.

4.2 Docker Compose and Docker Swarm

Compose is used for development and single-host deployment because it is simple and supports `depends_on` with health conditions. Swarm is used to show what a cluster adds: replicas, a virtual IP with load balancing, and automatic reconciliation. Both read the same YAML format, and both run exactly the same image, which is the central idea of containerisation: **build once, run anywhere**.

4.3 nginx and gunicorn

gunicorn is a pre-fork Python application server: a master process supervises three worker processes, each able to handle a request. It is not designed to face the internet directly, so nginx sits in front. nginx serves static files and uploads straight from the shared volumes (far faster than Python), limits the request body to 12 MB for photographs and passes the original host and scheme to Django through `X-Forwarded-*` headers.

4.4 GitHub Actions and Kubernetes

GitHub Actions runs the pipeline on every push; its *service containers* give the test job a real MySQL 8.4 database with a single YAML block. Kubernetes is the production target of the same image: the manifests in `deploy/k8s/` describe three replicas with readiness and liveness probes on `/healthz`, a zero-downtime rolling update, a horizontal pod autoscaler and a migration Job.

CHAPTER 5: SYSTEM DESIGN

5.1 Container Architecture

Figure 5.1 shows the containerised system as it runs under Docker Compose. Three containers share one user-defined bridge network. Only nginx publishes a port to the host; the web container only *exposes* port 8000 on the private network, and the database port is published on 3307 solely for inspection with a MySQL client.

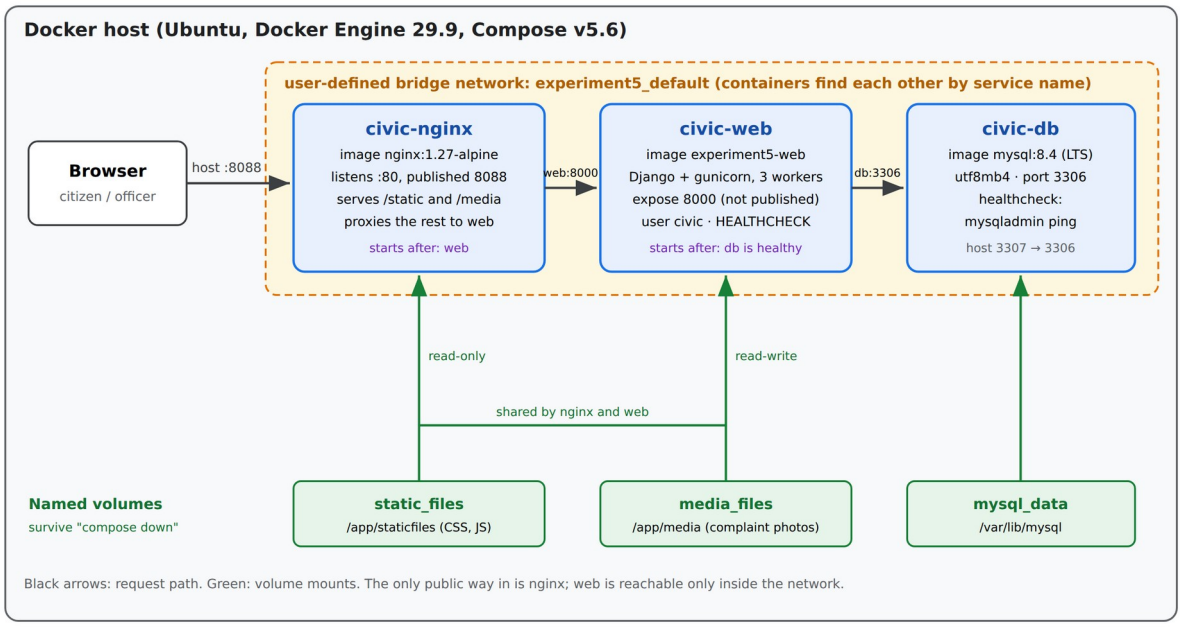


Figure 5.1 – Container architecture of CivicSense under Docker Compose

Table 5.1 – Services of the Compose stack

Service	Image	Ports	Volumes	Health / restart
db	mysql:8.4	3306 (host 3307)	mysql_data	mysqladmin ping every 10 s · unless-stopped
web	experiment5-web (built)	8000, not published	static_files, media_files (rw)	curl /healthz every 30 s · unless-stopped
nginx	nginx:1.27-alpine	80 (host 8088*)	static_files, media_files (ro), nginx.conf (ro)	unless-stopped

The project file publishes nginx on host port 8080. On the demonstration machine port 8080 was already used by Jenkins, so a small override file publishes it on 8088 instead (Appendix A); nothing else changes

5.2 Network and Volume Design

Compose creates the network `experiment5_default` (named after the project folder; the published repository fixes the project name to `civicsense`). Docker's embedded DNS resolves each service name to its container, so the application reaches the database as `db:3306` and nginx reaches the application as `web:8000`; no IP address appears anywhere in the

configuration. The measured addresses were 172.19.0.2 (db), 172.19.0.3 (web) and 172.19.0.4 (nginx).

Three named volumes hold everything that must outlive a container. `mysql_data` keeps the database across `docker compose down`. `static_files` is written by the web container (collectstatic) and read by nginx, and `media_files` holds the photographs that citizens upload. nginx mounts both read-only, so a compromised proxy cannot alter them.

5.3 Start-up Sequence

A multi-container system fails in subtle ways if its parts start in the wrong order: an application that starts before its database crashes, and a migration that runs twice at once corrupts the schema. Figure 5.2 shows how CivicSense makes the order explicit. Compose starts the web container only when the database is *healthy* (not merely *started*); the entrypoint then waits for MySQL a second time, migrates, collects the static files and finally replaces itself with gunicorn.

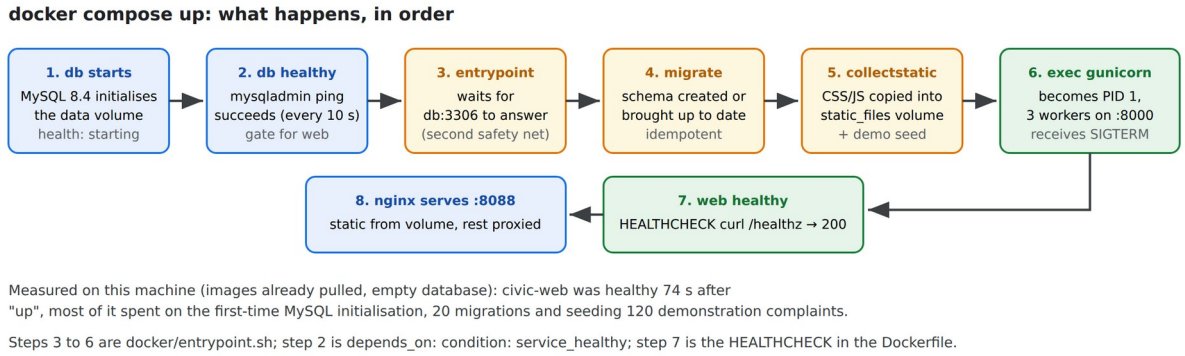


Figure 5.2 – Start-up sequence of the stack: health-gated dependencies and the entrypoint

5.4 Image Design: a Two-Stage Build

The Python package `mysqlclient` contains a C extension and needs `gcc`, `make` and the MySQL development headers to install. Installing those in the final image would make it larger and would leave a compiler inside a production container. The Dockerfile therefore uses two stages (Figure 5.3). The builder stage installs the toolchain and runs `pip install --prefix=/install`; the runtime stage starts again from a clean `python:3.12-slim` and copies only the `/install` folder. The measured builder stage is **698 MB**, the shipped image **435 MB** – 263 MB (38 %) smaller, with no build tools.

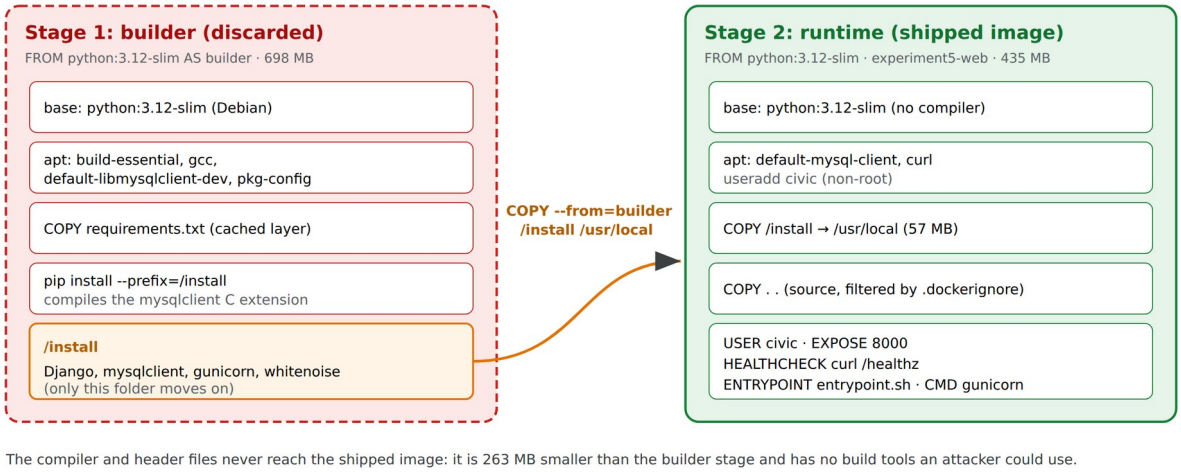


Figure 5.3 – Two-stage build: only the installed packages move to the runtime image

5.5 Data Model and Data Flow

Figure 5.4 shows the nine tables of CivicSense. Department, Category and Ward are master data; Complaint is the central table and also stores the decisions of the NLP engine; Attachment, ComplaintUpdate (the audit trail), Support and Feedback depend on it; TrainingSample stores every officer correction. Because the same migrations are applied by the container entrypoint, by the CI pipeline and by the Kubernetes Job, the schema is identical on a laptop, in the pipeline and in the cluster.

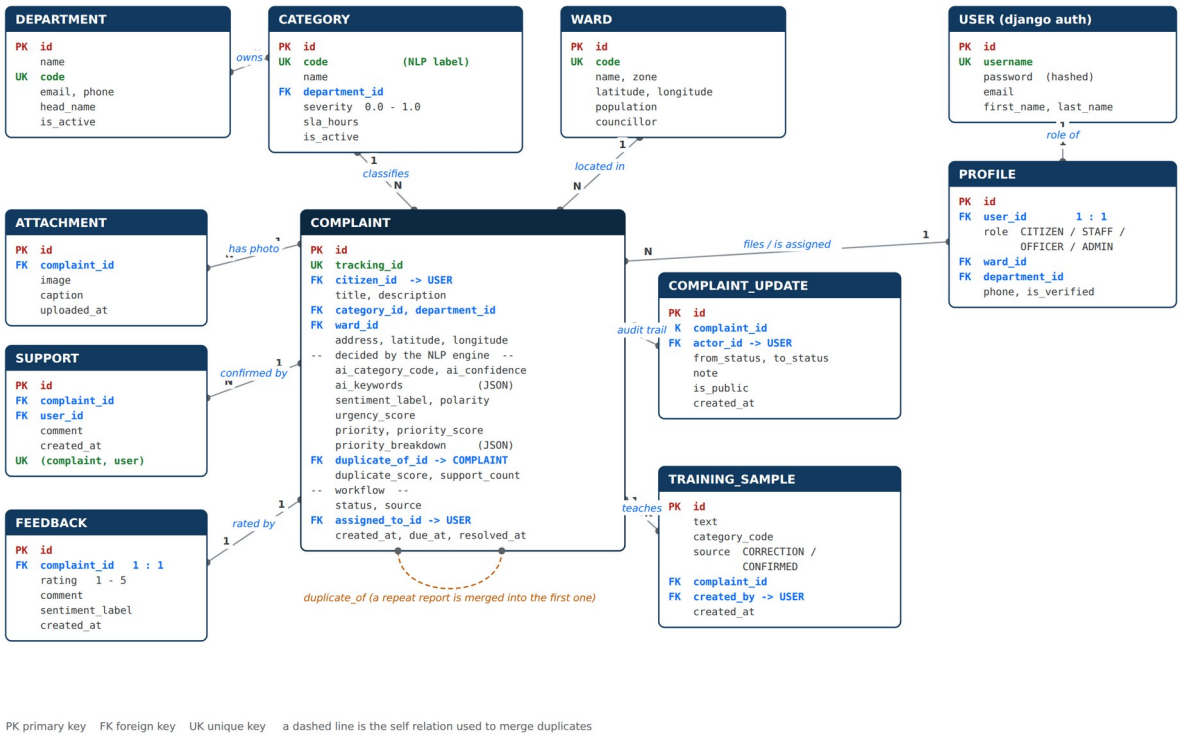


Figure 5.4 – Entity-relationship diagram of CivicSense

Figure 5.5 is the level-1 data flow diagram. Processes 1.0 to 6.0 are stateless and only the data stores D1 to D5 hold state. This separation is what allows the web tier to be replicated: any replica can serve any request because everything it needs is in MySQL or on the shared volumes.

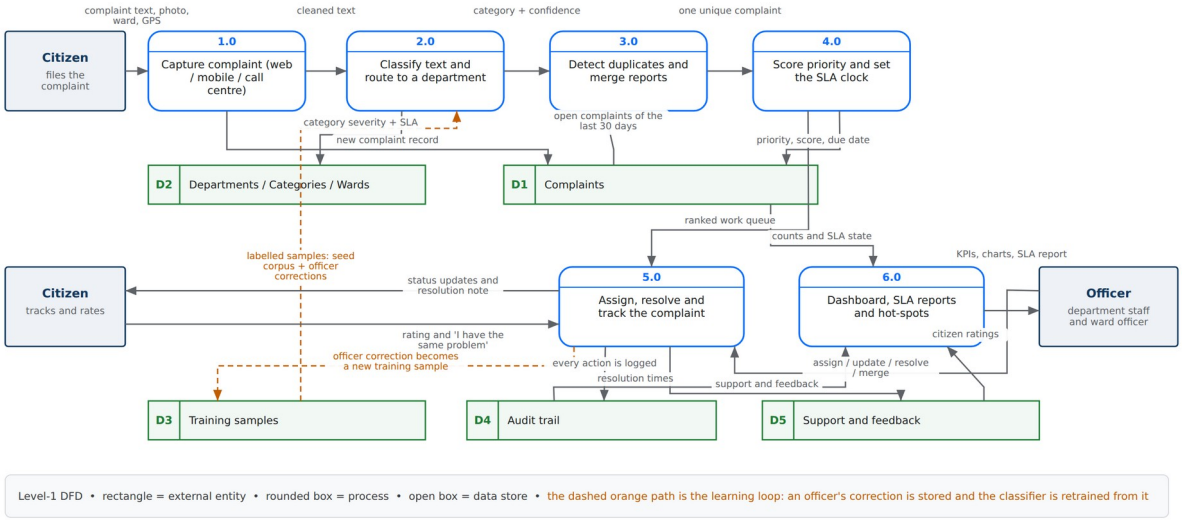


Figure 5.5 – Level-1 data flow diagram

5.6 Security Design

Table 5.2 – Security measures in the container design

Measure	Implementation
Non-root process	`useradd civic` and `USER civic`; `id` inside the container returns uid=1000(civic)
No build tools in production	Compiler and headers exist only in the discarded builder stage
Small build context	`.dockerignore` excludes .git, .venv, .env, db.sqlite3, media, docs and PDFs
No secrets in the image	Secret key and passwords injected as environment variables (Compose), Secrets (Kubernetes)
Minimal exposure	Only nginx is published; web is exposed only on the private network
Read-only mounts	nginx mounts the configuration, static and media volumes read-only
Least-privilege database user	The application user `civic` has rights only on the `civicsense` schema; it cannot even create the test database
Production settings	With DJANGO_DEBUG=0: secure cookies, HSTS, X-Frame-Options DENY, trusted proxy header; checked by `manage.py check --deploy` in CI

CHAPTER 6: IMPLEMENTATION STEPS

The project was implemented in the following order. Each step produced a file that is part of the repository, so the whole procedure can be repeated by anyone with Docker installed.

6.1 Step 1: Externalise the Configuration

Every value that changes between a laptop and a server – secret key, debug flag, allowed hosts, trusted origins and database credentials – was moved into environment variables read by `civicsense/settings.py`. When `DB_ENGINE` is not `mysql`, SQLite is used, so the project still runs with `manage.py runserver` and no database installation.

6.2 Step 2: Add a Health Endpoint

A four-line view `/healthz` returning `{"status": "ok"}` was added. It is used by the Docker HEALTHCHECK, by the Compose start-up order, by the CI smoke test, by Swarm and by both Kubernetes probes.

6.3 Step 3: Write the Dockerfile and .dockerignore

The two-stage Dockerfile described in Section 5.4 was written, with `requirements.txt` copied before the source code so that the expensive package layer is re-used whenever only the code changes. The image is built with:

```
docker build -t civicsense:latest .          # or: docker compose build
```

6.4 Step 4: Write the Entrypoint Script

`docker/entrypoint.sh` waits for MySQL, applies migrations, collects static files, optionally seeds demonstration data and finally executes the command given in CMD with `exec "$@"`, so that gunicorn becomes PID 1 and receives stop signals directly.

6.5 Step 5: Describe the Stack with Docker Compose

`docker-compose.yml` defines the three services, their environment, the health check of MySQL, the `depends_on: condition: service_healthy` gate, the named volumes and the published proxy port. The whole platform is started and inspected with:

```
docker compose up -d --build        # build the image and start db, web and nginx
docker compose exec web python manage.py seed_demo --reset   # demo data, once
docker compose ps                   # status and health of the three containers
docker compose logs -f web          # follow the application log
docker compose down                 # stop and remove containers; volumes are kept
```

6.6 Step 6: Configure nginx as Reverse Proxy

`docker/nginx.conf` defines an upstream `web:8000`, serves `/static/` and `/media/` from the volumes, forwards the `Host` and `X-Forwarded-*` headers and raises the body limit to 12 MB for photographs.

6.7 Step 7: Verify the Stack

The running stack was verified with `curl` against `/healthz`, `docker inspect` of the health state, `docker logs`, `docker network inspect`, `docker stats`, a SQL query in the database container and the full test suite run inside the web container (Chapter 8).

6.8 Step 8: Continuous Integration

`.github/workflows/ci.yml` defines two jobs. The *test* job starts a MySQL 8.4 service container, installs the dependencies, runs flake8, `manage.py check --deploy` and the 39 tests against MySQL. The *docker* job runs only if *test* succeeded (`needs: test`); it builds the image tagged with the commit SHA, starts it and polls `/healthz` up to 30 times, printing the container logs if it never becomes healthy (Figure 6.1).

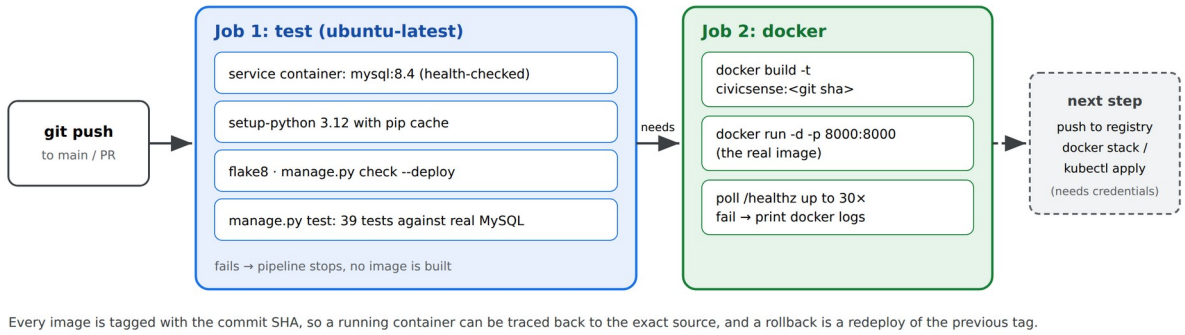


Figure 6.1 – CI pipeline: no green tests, no image; every image is smoke-tested

6.9 Step 9: Deploy on Docker Swarm

A stack file, `docker-stack.yml`, was written for Swarm mode (Section 7.8). The swarm was initialised on the local machine, the stack was deployed, and the web service was scaled, crashed, updated and rolled back while traffic was being sent to it (Figure 6.2).

```
docker swarm init --advertise-addr 127.0.0.1
docker stack deploy -c docker-stack.yml civicsense
docker service ls
docker service scale civicsense_web=5
docker service update --env-add DJANGO_LOG_LEVEL=WARNING civicsense_web
docker service rollback civicsense_web
docker stack rm civicsense && docker swarm leave --force
```

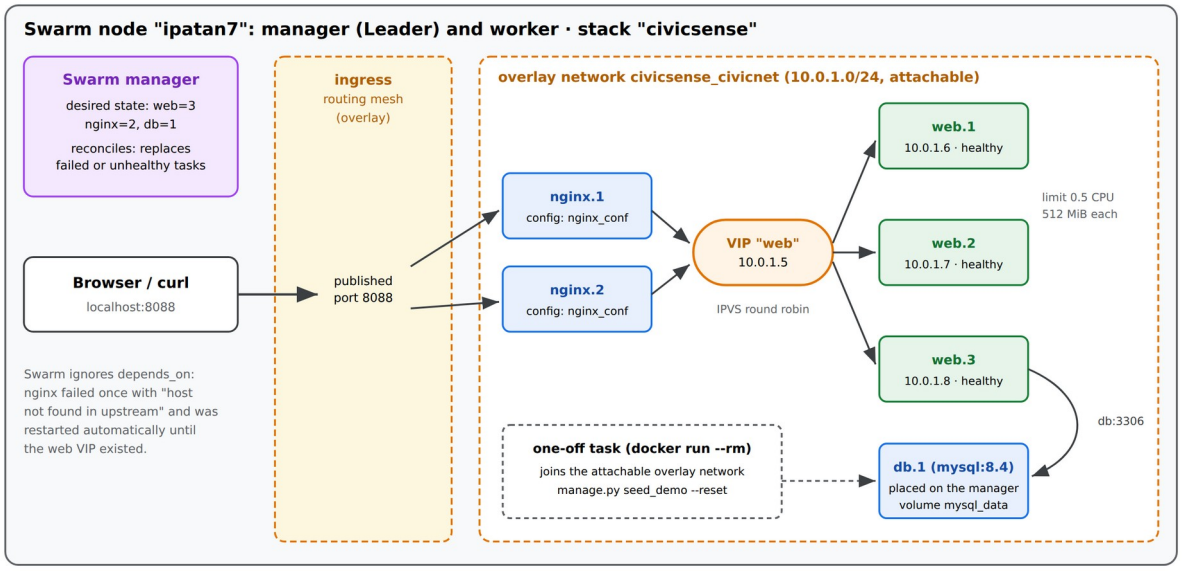


Figure 6.2 – CivicSense deployed as a Docker Swarm stack on one node

6.10 Step 10: Kubernetes Manifests

Finally the same image was described for Kubernetes in `deploy/k8s/`: a Deployment of three replicas with probes and resource limits, a ClusterIP Service, a HorizontalPodAutoscaler (2–8 pods at 70 % CPU), a migration Job, a ConfigMap and a Secret template. Applying them is a single `kubectl apply -f deploy/k8s/` once the image is pushed to a registry.

CHAPTER 7: IMPLEMENTATION AND CODE EXPLANATION

7.1 Project Structure

Repository layout (github.com/iPatan7/civicsense-docker)

```
civicsense-docker/
├── Dockerfile               two-stage production image
├── .dockerignore            keeps .git, .env, db.sqlite3, media out of the image
├── docker-compose.yml       db + web + nginx on one host
├── docker-compose.demo.yml  publishes nginx on 8088 when 8080 is busy
├── docker-stack.yml         the same system as a Docker Swarm stack
├── docker/
│   ├── entrypoint.sh        wait for MySQL, migrate, collectstatic, exec gunicorn
│   └── nginx.conf           reverse proxy, static and media files
├── .github/workflows/ci.yml test on MySQL, build and smoke-test the image
├── deploy/k8s/              Deployment, Service, HPA, Job, ConfigMap, Secret
├── civicsense/              settings (twelve-factor), URLs, WSGI
├── accounts/ complaints/ dashboard/   the Django applications
├── nlpengine/               classifier, duplicate detection, sentiment, priority
├── report/                  this report and the one-page case study (PDF)
└── requirements.txt         Django, mysqlclient, gunicorn, whitenoise, python-dotenv
```

7.2 Twelve-Factor Configuration

The settings read every environment-specific value from the environment. The same image therefore runs in development, in Compose, in Swarm and in Kubernetes without being rebuilt.

civicsense/settings.py (excerpt, simplified)

```
SECRET_KEY    = env("DJANGO_SECRET_KEY", "dev-only-insecure-key-change-me")
DEBUG         = env_bool("DJANGO_DEBUG", True)
ALLOWED_HOSTS = [h for h in env("DJANGO_ALLOWED_HOSTS", "*").split(",") if h]
CSRF_TRUSTED_ORIGINS = [o for o in env("DJANGO_CSRF_TRUSTED_ORIGINS", "").split(",")
                        if o]

if env("DB_ENGINE", "sqlite") == "mysql":            # deployment
    DATABASES = {"default": {"ENGINE": "django.db.backends.mysql",
        "NAME": env("DB_NAME"), "USER": env("DB_USER"),
        "PASSWORD": env("DB_PASSWORD"),
        "HOST": env("DB_HOST"), "PORT": env("DB_PORT", "3306"),
        "OPTIONS": {"charset": "utf8mb4"}}}
else:                                                # development, zero setup
    DATABASES = {"default": {"ENGINE": "django.db.backends.sqlite3",
                             "NAME": BASE_DIR / "db.sqlite3"}}

if not DEBUG:                                        # switched on by the environment
    SESSION_COOKIE_SECURE = CSRF_COOKIE_SECURE = True
    SECURE_HSTS_SECONDS = 31536000
    SECURE_PROXY_SSL_HEADER = ("HTTP_X_FORWARDED_PROTO", "https")
```

7.3 The Health Endpoint

complaints/views.py and civicsense/urls.py

```
def healthz(request):
    """Liveness probe for Docker / Kubernetes."""
    return JsonResponse({"status": "ok"})

path("healthz", healthz, name="healthz")
```

The endpoint deliberately does not need a login and does not touch the database, so a slow query can never make a healthy container look dead.

7.4 The Dockerfile

Dockerfile

```
# ------------------------------------------------------------------------
# CivicSense - production image
#
# A two stage build: the first stage compiles the Python wheels (mysqlclient
# needs a C compiler), the second stage copies only the installed packages, so
# the shipped image carries no build tools.
# ------------------------------------------------------------------------
FROM python:3.12-slim AS builder

ENV PYTHONDONTWRITEBYTECODE=1 PYTHONUNBUFFERED=1
WORKDIR /build

RUN apt-get update && apt-get install -y --no-install-recommends \
        build-essential default-libmysqlclient-dev pkg-config \
    && rm -rf /var/lib/apt/lists/*

COPY requirements.txt .
RUN pip install --no-cache-dir --prefix=/install -r requirements.txt



FROM python:3.12-slim

ENV PYTHONDONTWRITEBYTECODE=1 \
    PYTHONUNBUFFERED=1 \
    DJANGO_SETTINGS_MODULE=civicsense.settings
WORKDIR /app

RUN apt-get update && apt-get install -y --no-install-recommends \
        default-mysql-client curl \
    && rm -rf /var/lib/apt/lists/* \
    && useradd --create-home --shell /bin/bash civic

COPY --from=builder /install /usr/local
COPY . .

RUN mkdir -p /app/staticfiles /app/media && chown -R civic:civic /app
USER civic

EXPOSE 8000
HEALTHCHECK --interval=30s --timeout=5s --start-period=20s --retries=3 \
    CMD curl -fsS http://localhost:8000/healthz || exit 1

ENTRYPOINT ["/app/docker/entrypoint.sh"]
CMD ["gunicorn", "civicsense.wsgi:application", \
     "--bind", "0.0.0.0:8000", "--workers", "3", "--timeout", "60", \
     "--access-logfile", "-"]
```

Table 7.1 – Dockerfile instructions explained

Instruction	Purpose
FROM python:3.12-slim AS builder	Starts the throw-away build stage from the slim Debian image of Python
apt-get install build-essential …	Compiler and MySQL headers needed to build mysqlclient; `rm -rf /var/lib/apt/lists/*` keeps the layer small

Instruction	Purpose
COPY requirements.txt + pip install --prefix=/install	Installs all packages into one folder; cached until requirements.txt changes
FROM python:3.12-slim (second)	Starts the shipped image again from a clean base
ENV PYTHONDONTWRITEBYTECODE / PYTHONUNBUFFERED	No .pyc files in the image; logs reach `docker logs` immediately
apt-get install default-mysql-client curl	`mysqladmin` for the wait loop, `curl` for the health check
useradd civic … USER civic	The application never runs as root
COPY --from=builder /install /usr/local	Brings over only the installed packages (57 MB layer)
EXPOSE 8000	Documents the port gunicorn listens on
HEALTHCHECK … curl /healthz	Docker marks the container healthy or unhealthy every 30 s
ENTRYPOINT / CMD	The entrypoint prepares the release; CMD (gunicorn) can be replaced for one-off tasks

.dockerignore

```
.git
.venv
venv
db.sqlite3
media
staticfiles
docs
*.pdf
__pycache__
*.pyc
.env
```

7.5 The Entrypoint Script

docker/entrypoint.sh

```
#!/bin/sh
# Container start-up: wait for MySQL, apply the migrations, collect the static
# files and then hand over to gunicorn (or to whatever CMD was given).
set -e

if [ "${DB_ENGINE}" = "mysql" ]; then
  echo "Waiting for MySQL at ${DB_HOST}:${DB_PORT} ..."
  until mysqladmin ping -h"${DB_HOST}" -P"${DB_PORT}" --silent 2>/dev/null; do
    sleep 2
  done
  echo "MySQL is up."
fi

python manage.py migrate --no-input
python manage.py collectstatic --no-input --clear >/dev/null

if [ "${SEED_DEMO_DATA}" = "1" ]; then
  echo "Seeding demonstration data ..."
  python manage.py seed_demo || true
```

```
fi

exec "$@"
```

`set -e` stops the container if any step fails instead of serving a half-migrated database. The final `exec "$@"` replaces the shell with gunicorn, so gunicorn becomes PID 1: when Docker stops the container it sends SIGTERM to gunicorn itself, which finishes the requests in progress before exiting. Section 8.5 shows a defect in the optional seeding step and how it was fixed.

7.6 The Compose File

docker-compose.yml

```
# One command brings up the whole platform:  docker compose up --build
#
#   db    - MySQL 8, the production database of the project
#   web   - Django served by gunicorn
#   nginx - serves /static and /media and proxies everything else to web
# A fixed project name keeps the volume names (civicsense_mysql_data, ...) the same on
# every machine, so the swarm stack "civicsense" finds and re-uses the same data.
name: civicsense

services:
  db:
    image: mysql:8.4   # 8.4 LTS: required by Django 6.x
    container_name: civic-db
    restart: unless-stopped
    environment:
      MYSQL_DATABASE: civicsense
      MYSQL_USER: civic
      MYSQL_PASSWORD: civic
      MYSQL_ROOT_PASSWORD: rootpass
    command: --character-set-server=utf8mb4 --collation-server=utf8mb4_unicode_ci
    volumes:
      - mysql_data:/var/lib/mysql
    healthcheck:
      test: ["CMD", "mysqladmin", "ping", "-h", "localhost", "-uroot", "-prootpass"]
      interval: 10s
      timeout: 5s
      retries: 10
    ports:
      - "3307:3306"        # host port 3307 keeps a local MySQL untouched

  web:
    build: .
    container_name: civic-web
    restart: unless-stopped
    environment:
      DJANGO_SECRET_KEY: ${DJANGO_SECRET_KEY:-compose-dev-key}
      DJANGO_DEBUG: "0"
      DJANGO_ALLOWED_HOSTS: localhost,127.0.0.1,web,nginx
      # nginx passes the Host header without the port, so the published port has
      # to be listed here or every login is rejected by Django's CSRF origin check.
      DJANGO_CSRF_TRUSTED_ORIGINS: http://localhost:8080,http://127.0.0.1:8080
      DB_ENGINE: mysql
      DB_NAME: civicsense
      DB_USER: civic
      DB_PASSWORD: civic
      DB_HOST: db
      DB_PORT: "3306"
      # Demo data is loaded once with:
      #   docker compose exec web python manage.py seed_demo --reset
      # Seeding on every start would add the demo complaints again after each restart.
      SEED_DEMO_DATA: "0"
    volumes:
      - static_files:/app/staticfiles
```

```
      - media_files:/app/media
    depends_on:
      db:
        condition: service_healthy
    expose:
      - "8000"

  nginx:
    image: nginx:1.27-alpine
    container_name: civic-nginx
    restart: unless-stopped
    ports:
      - "8080:80"
    volumes:
      - ./docker/nginx.conf:/etc/nginx/conf.d/default.conf:ro
      - static_files:/app/staticfiles:ro
      - media_files:/app/media:ro
    depends_on:
      - web

volumes:
  mysql_data:
  static_files:
  media_files:
```

Three details matter. `depends_on` with `condition: service_healthy` waits for MySQL to *answer*, not just to *start*. The web service uses `expose` instead of `ports`, so it can only be reached through nginx. And the MySQL host port is 3307, so a MySQL server already installed on the host is not disturbed.

7.7 The nginx Configuration

docker/nginx.conf

```
upstream civic_web {
    server web:8000;
}

server {
    listen 80;
    server_name _;
    client_max_body_size 12M;          # complaint photographs

    # Static files and citizen uploads are served by nginx, never by Django.
    location /static/ { alias /app/staticfiles/; expires 30d; access_log off; }
    location /media/  { alias /app/media/;      expires 7d; }

    location /healthz {
        proxy_pass http://civic_web;
        proxy_set_header Host $host;          # without this Django sees the
        proxy_set_header X-Real-IP $remote_addr;   # upstream name as the host
        access_log off;
    }

    location / {
        proxy_pass http://civic_web;
        proxy_set_header Host              $host;
        proxy_set_header X-Real-IP         $remote_addr;
        proxy_set_header X-Forwarded-For   $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
        proxy_redirect off;
    }
}
```

7.8 The Swarm Stack File

The stack file runs the same image as replicated services. The web service gets three replicas, a start-first rolling update with automatic rollback, a restart policy and CPU/memory limits; nginx gets two replicas published through the ingress routing mesh, and its configuration is distributed as a swarm **config** instead of a bind mount. Seeding is disabled because in a cluster it belongs to a one-off task, never to a restart.

docker-stack.yml

```
# CivicSense on Docker Swarm:  docker stack deploy -c docker-stack.yml civicsense
# Build the image first with "docker compose build". nginx is published on port 8088.
#
# The same image as docker-compose.yml, but run as replicated swarm services:
#   web   - 3 replicas behind the swarm's virtual IP, rolling updates, self-healing
#   nginx - 2 replicas published through the ingress routing mesh on port 8088
#   db    - 1 replica pinned to the manager node (it owns the data volume)
# The stack is called "civicsense" like the Compose project, so its volumes
# (civicsense_mysql_data, ...) are the ones docker compose created and the data carries
over.
services:
  db:
    image: mysql:8.4
    command: --character-set-server=utf8mb4 --collation-server=utf8mb4_unicode_ci
    environment:
      MYSQL_DATABASE: civicsense
      MYSQL_USER: civic
      MYSQL_PASSWORD: civic
      MYSQL_ROOT_PASSWORD: rootpass
    volumes:
      - mysql_data:/var/lib/mysql
    networks: [civicnet]
    healthcheck:
      test: ["CMD", "mysqladmin", "ping", "-h", "localhost", "-uroot", "-prootpass"]
      interval: 10s
      timeout: 5s
      retries: 10
    deploy:
      replicas: 1
      placement:
        constraints: ["node.role == manager"]
      restart_policy: {condition: on-failure}


  web:
    image: civicsense-web:latest   # built by "docker compose build" (project name
civicsense)
    environment:
      DJANGO_SECRET_KEY: swarm-demo-key
      DJANGO_DEBUG: "0"
      DJANGO_ALLOWED_HOSTS: localhost,127.0.0.1,web,nginx
      DJANGO_CSRF_TRUSTED_ORIGINS: http://localhost,http://127.0.0.1
      DB_ENGINE: mysql
      DB_NAME: civicsense
      DB_USER: civic
      DB_PASSWORD: civic
      DB_HOST: db
      DB_PORT: "3306"
      SEED_DEMO_DATA: "0"          # seeding is a one-off task, never part of a restart
    volumes:
      - static_files:/app/staticfiles
      - media_files:/app/media
    networks: [civicnet]
    deploy:
      replicas: 3
      update_config:
        parallelism: 1             # one replica at a time
        delay: 10s
        order: start-first         # new task healthy before the old one stops
        failure_action: rollback
        monitor: 30s
      rollback_config:
        parallelism: 1
```

```
        order: start-first
      restart_policy:
        condition: any
        delay: 5s
      resources:
        limits: {cpus: "0.50", memory: 512M}
        reservations: {cpus: "0.10", memory: 128M}

  nginx:
    image: nginx:1.27-alpine
    ports:
      - "8088:80"                 # ingress routing mesh
    configs:
      - source: nginx_conf
        target: /etc/nginx/conf.d/default.conf
    volumes:
      - static_files:/app/staticfiles:ro
      - media_files:/app/media:ro
    networks: [civicnet]
    deploy:
      replicas: 2
      restart_policy: {condition: any}

networks:
  civicnet:
    driver: overlay
    attachable: true              # lets one-off "docker run" tasks join it

volumes:
  mysql_data:
  static_files:
  media_files:

configs:
  nginx_conf:
    file: ./docker/nginx.conf
```

7.9 The CI Workflow

.github/workflows/ci.yml

```
# Continuous integration for CivicSense.
#
# Every push runs the same three steps a reviewer would run by hand: the Django
# system checks, the test suite against a real MySQL 8 service, and a Docker
# build, so a broken image can never reach the registry.
name: CivicSense CI

on:
  push:
    branches: [main]
  pull_request:

jobs:
  test:
    runs-on: ubuntu-latest

    services:
      mysql:
        image: mysql:8.4
        env:
          MYSQL_DATABASE: civicsense
          MYSQL_USER: civic
          MYSQL_PASSWORD: civic
          MYSQL_ROOT_PASSWORD: rootpass
        ports: ["3306:3306"]
        options: >-
          --health-cmd="mysqladmin ping -h localhost -uroot -prootpass"
          --health-interval=10s --health-timeout=5s --health-retries=10

    steps:
```

```
      - uses: actions/checkout@v4

      - uses: actions/setup-python@v5
        with:
          python-version: "3.12"
          cache: pip

      - name: Install system and Python dependencies
        run: |
          sudo apt-get update
          sudo apt-get install -y default-libmysqlclient-dev pkg-config
          pip install -r requirements.txt
          pip install flake8

      - name: Lint (style only, no failing build on warnings)
        run: flake8 --max-line-length=100 --exclude=migrations . || true

      - name: Django system checks
        env:
          DJANGO_SECRET_KEY: ci-secret
          DJANGO_DEBUG: "0"
          DJANGO_ALLOWED_HOSTS: localhost
        run: python manage.py check --deploy --fail-level ERROR

      - name: Run the test suite against MySQL
        env:
          DJANGO_SECRET_KEY: ci-secret
          DB_ENGINE: mysql
          DB_NAME: civicsense
          DB_USER: root
          DB_PASSWORD: rootpass
          DB_HOST: 127.0.0.1
          DB_PORT: "3306"
        run: python manage.py test --verbosity 2

  docker:
    runs-on: ubuntu-latest
    needs: test
    steps:
      - uses: actions/checkout@v4
      - name: Build the production image
        run: docker build -t civicsense:${{ github.sha }} .
      - name: Smoke-test the image
        run: |
          docker run -d --name civic-smoke -p 8000:8000 \
            -e DJANGO_SECRET_KEY=smoke -e DJANGO_ALLOWED_HOSTS='*' \
            civicsense:${{ github.sha }}
          for i in $(seq 1 30); do
            if curl -fsS http://localhost:8000/healthz; then exit 0; fi
            sleep 2
          done
          docker logs civic-smoke
          exit 1
```

7.10 The Kubernetes Manifests

deploy/k8s/deployment.yaml

```
# Three stateless replicas of the Django application.  Because every request
# is stateless (the session lives in the database) the pods can be added or
# removed at any time, which is what makes horizontal scaling possible.
apiVersion: apps/v1
kind: Deployment
metadata:
  name: civicsense-web
  labels: {app: civicsense}
spec:
  replicas: 3
  selector:
    matchLabels: {app: civicsense}
  strategy:
    type: RollingUpdate            # no downtime while a new version rolls out
```

```
    rollingUpdate: {maxSurge: 1, maxUnavailable: 0}
  template:
    metadata:
      labels: {app: civicsense}
    spec:
      containers:
        - name: web
          image: ghcr.io/example/civicsense:latest
          ports: [{containerPort: 8000}]
          envFrom:
            - configMapRef: {name: civicsense-config}
            - secretRef: {name: civicsense-secret}
          readinessProbe:
            httpGet: {path: /healthz, port: 8000}
            initialDelaySeconds: 10
            periodSeconds: 10
          livenessProbe:
            httpGet: {path: /healthz, port: 8000}
            initialDelaySeconds: 30
            periodSeconds: 20
          resources:
            requests: {cpu: "150m", memory: "256Mi"}
            limits:   {cpu: "700m", memory: "768Mi"}
---
apiVersion: v1
kind: Service
metadata:
  name: civicsense-web
spec:
  selector: {app: civicsense}
  ports:
    - port: 80
      targetPort: 8000
  type: ClusterIP
---
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: civicsense-web
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: civicsense-web
  minReplicas: 2
  maxReplicas: 8
  metrics:
    - type: Resource
      resource:
        name: cpu
        target: {type: Utilization, averageUtilization: 70}
```

The readiness probe answers "may traffic be sent to this pod?" and the liveness probe answers "is this pod stuck and should it be restarted?". With `maxUnavailable: 0` a new pod must become ready before an old one is removed, which is the Kubernetes equivalent of the start-first update order used in Swarm.

CHAPTER 8: TESTING AND RESULTS

8.1 Testing Strategy

The project was tested at four levels, all against the real deployment rather than against `runserver`:

- **Image level** – build success, image size, layers, user, health check;
- **Stack level** – start-up order, health states, networking, volumes, logs and resource usage under Docker Compose;
- **Application level** – the 39 automated Django tests executed inside the running container against MySQL 8.4;
- **Resilience level** – crash recovery under Compose, and replicas, load balancing, scaling, self-healing, rolling update and rollback under Docker Swarm, each while traffic was being sent and every response counted.

8.2 Build Results

The image was built from scratch in 187 s (Figure 8.1). The runtime packages installed with apt (MySQL client and curl) account for 153 MB and the Python packages for 57 MB; the application source adds only about 1 MB, so a code change produces a new image that differs from the previous one by about 2 MB.

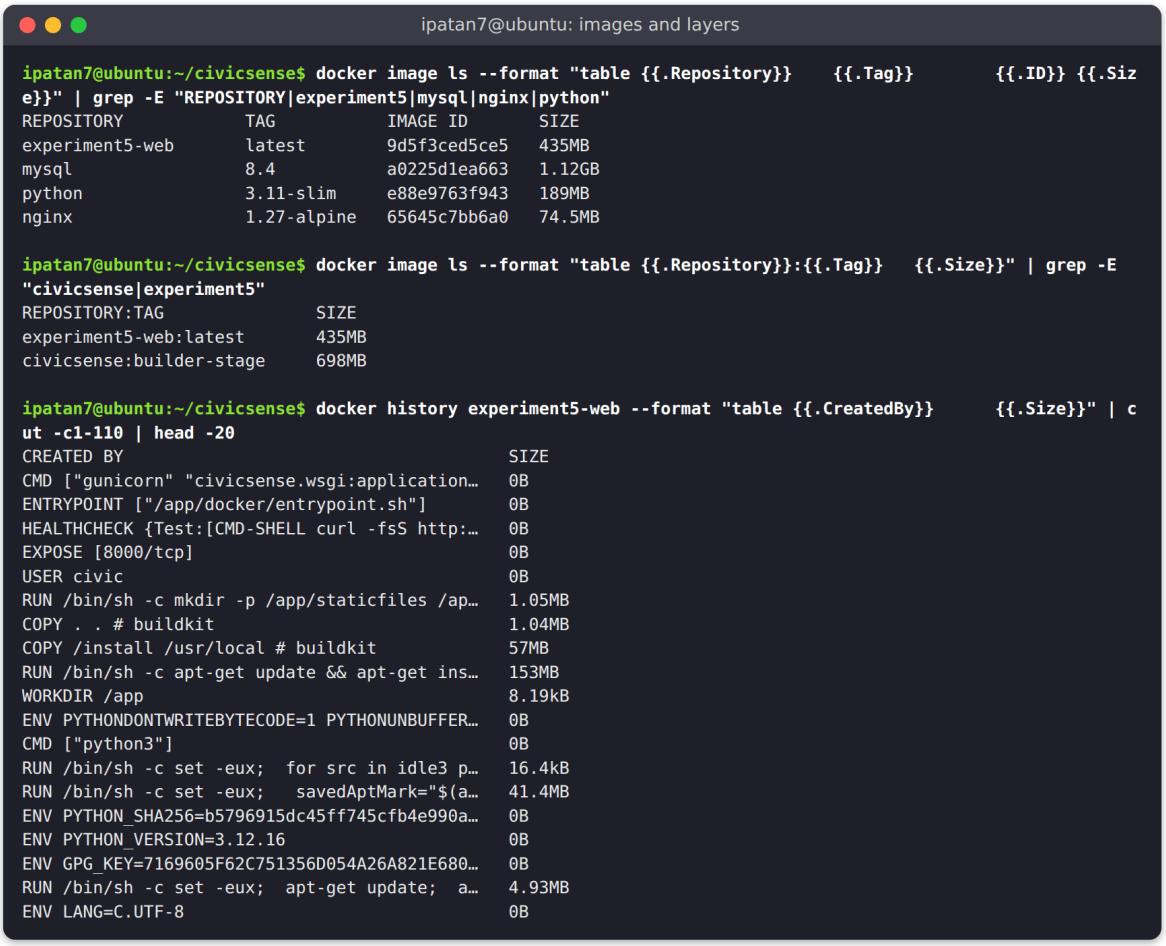


Figure 8.1 – Image sizes (runtime 435 MB vs builder 698 MB) and the layers of the image

Table 8.1 – Build results

Measurement	Result
Cold build (including base image download)	187 s
Builder stage / shipped image	698 MB / 435 MB (38 % smaller)
Largest layers	apt runtime 153 MB · Python packages 57 MB · source 1.04 MB
Build context exclusions	.git, .venv, .env, db.sqlite3, media, docs, PDFs

8.3 Stack Bring-up and Health

`docker compose up` was run with an empty database. The web container became healthy **74 s** later, most of which was spent on the first initialisation of MySQL, 20 migrations and seeding 120 demonstration complaints. A warm restart with an existing database took **18 s**. Figure 8.2 shows the three containers, the health states and the portal answering through nginx.

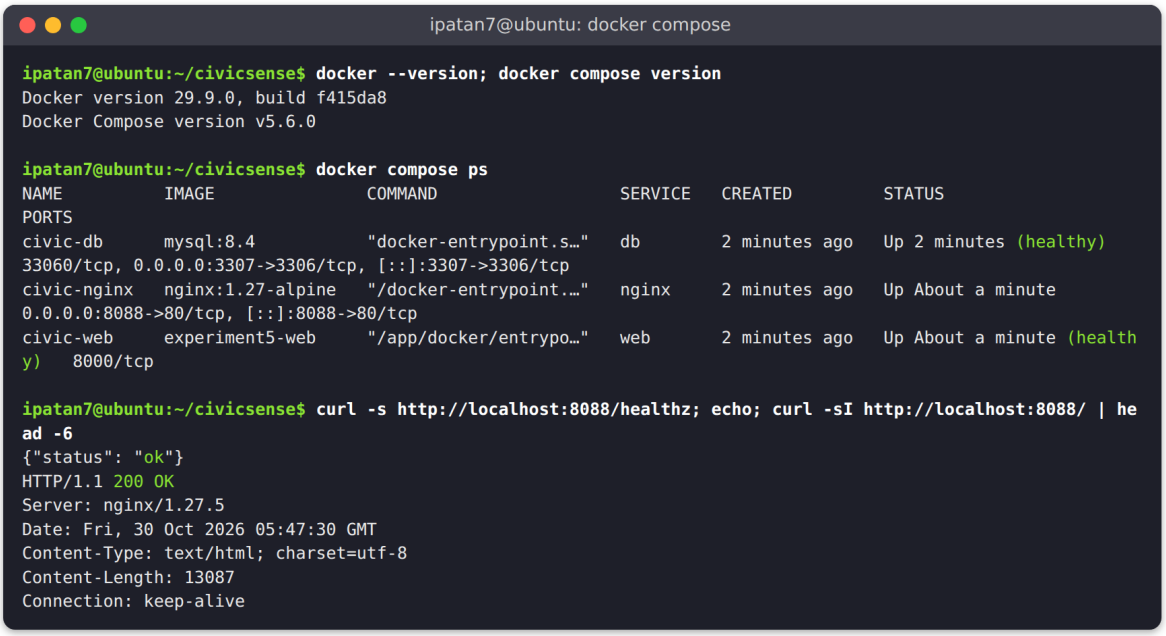


Figure 8.2 – docker compose ps: database and web healthy, only nginx published

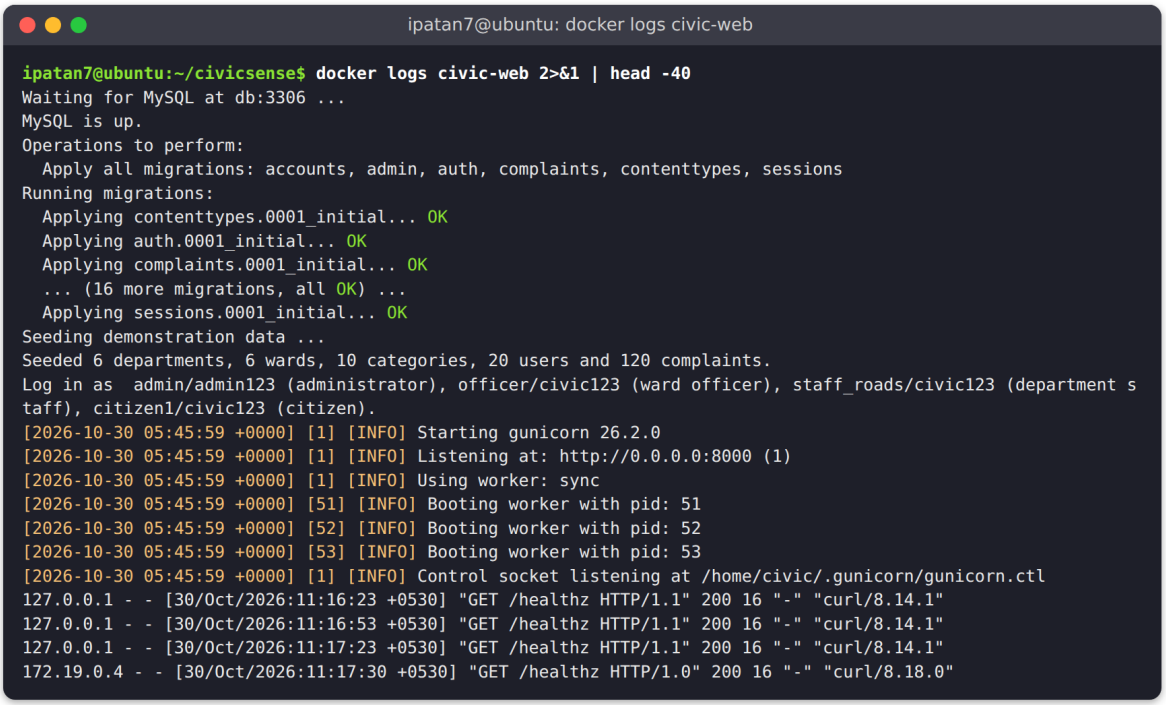


Figure 8.3 – Container log: wait for MySQL, migrate, seed, then gunicorn with three workers

The health log (Figure 8.4) shows the health check doing its job: the first two probes failed with *connection refused* while migrations were still running, and the container was reported healthy only once `/healthz` answered. The same figure confirms that the process runs as `civic` (uid 1000) and that the restart policy is `unless-stopped`.

ipatan7@ubuntu: health check and user

```
ipatan7@ubuntu:~/civicsense$ docker inspect --format "{{json .State.Health}}" civic-web | python3 -m json.tool
| head -30
{
    "Status": "healthy",
    "FailingStreak": 0,
    "Log": [
        {
            "Start": "2026-10-30T11:15:48.712934426+05:30",
            "End": "2026-10-30T11:15:48.757844654+05:30",
            "ExitCode": 1,
            "Output": "curl: (7) Failed to connect to localhost port 8000 after 0 ms: Could not connect to ser
ver\n"
        },
        {
            "Start": "2026-10-30T11:15:53.45922244+05:30",
            "End": "2026-10-30T11:15:53.504145257+05:30",
            "ExitCode": 1,
            "Output": "curl: (7) Failed to connect to localhost port 8000 after 0 ms: Could not connect to ser
ver\n"
        },
        {
            "Start": "2026-10-30T11:16:23.504961064+05:30",
            "End": "2026-10-30T11:16:23.588590689+05:30",
            "ExitCode": 0,
            "Output": "{\"status\": \"ok\"}"
        ...
    ]
}

ipatan7@ubuntu:~/civicsense$ docker inspect civic-web --format "Image={{.Config.Image}} User={{.Config.User}}
Restart={{.HostConfig.RestartPolicy.Name}} Health={{.State.Health.Status}} Started={{.State.StartedAt}}"
Image=experiment5-web User=civic Restart=unless-stopped Health=healthy Started=2026-10-30T05:45:33.458421425Z

ipatan7@ubuntu:~/civicsense$ docker exec civic-web id; docker top civic-web -o pid,user,args | cut -c1-110
uid=1000(civic) gid=1000(civic) groups=1000(civic)
PID                 USER                COMMAND
24791               ipatan7             /usr/local/bin/python3.12 /usr/local/bin/gunicorn civicsense.wsgi:appl
25808               ipatan7             /usr/local/bin/python3.12 /usr/local/bin/gunicorn civicsense.wsgi:appl
25809               ipatan7             /usr/local/bin/python3.12 /usr/local/bin/gunicorn civicsense.wsgi:appl
25810               ipatan7             /usr/local/bin/python3.12 /usr/local/bin/gunicorn civicsense.wsgi:appl
```

Figure 8.4 – Health-check history, restart policy and the non-root user

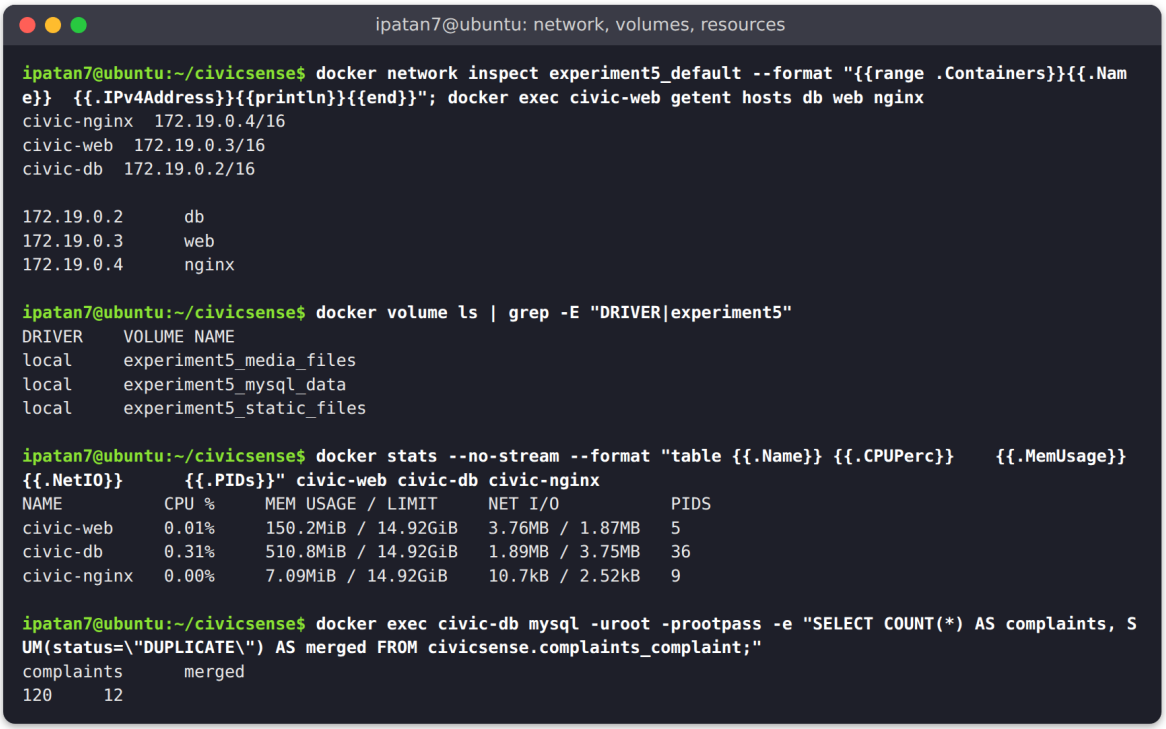


Figure 8.5 – Service-name DNS on the bridge network, named volumes, resource usage and data in MySQL

8.4 Automated Tests inside the Container

The 39 tests cover the NLP engine (classification of all ten services, sentiment with negation, emergency detection, duplicate detection by text, place and time), the service layer (routing, SLA clock, merging, support, corrections, status flow) and the web layer (access control by role, the filing form, the live-analysis API). Running them inside the container proves that the image itself – not just the developer's laptop – contains a working application.

The first attempt failed with *Access denied for user 'civic' to database 'test_civicsense'*. This is correct behaviour: the application's database account is limited to its own schema and is not allowed to create databases. The tests were therefore run with the administrative account, exactly as the CI pipeline does, and **all 39 tests passed in 85.8 s against MySQL 8.4** (Figure 8.6).

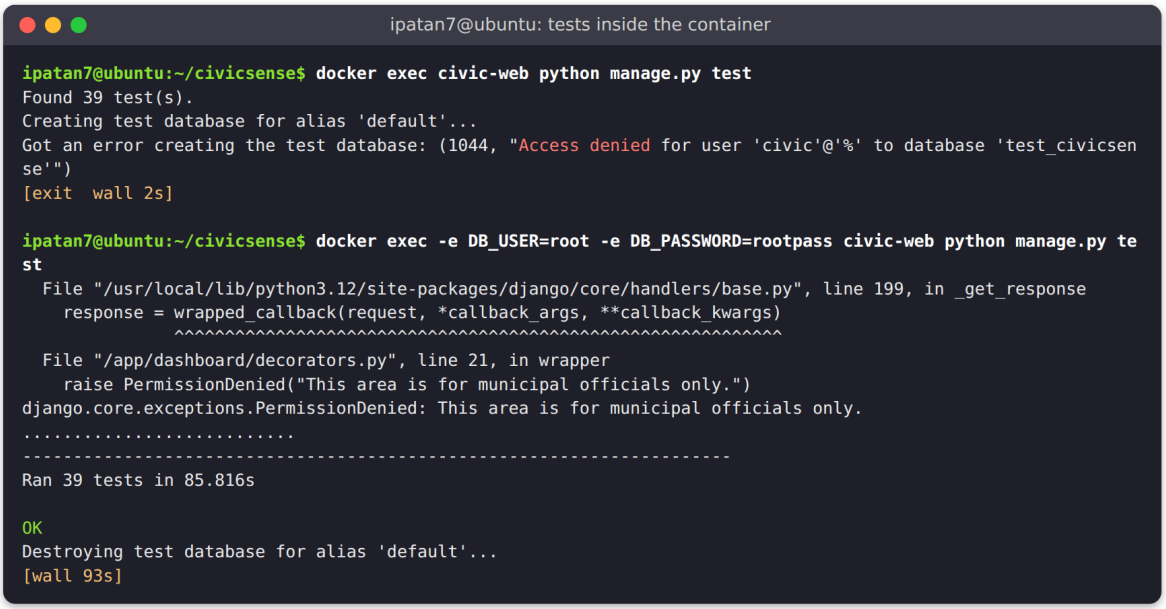


Figure 8.6 – Least privilege refuses the test database; with the admin account all 39 tests pass

8.5 Self-Healing with a Restart Policy

To simulate a crash, the gunicorn master process of the running web container was killed from the host with SIGKILL. Docker recorded exit code 137, restarted the container **within 1.1 s** because of `restart: unless-stopped`, and the container was healthy again **11.3 s** after the kill (Figure 8.7). No human action was needed.

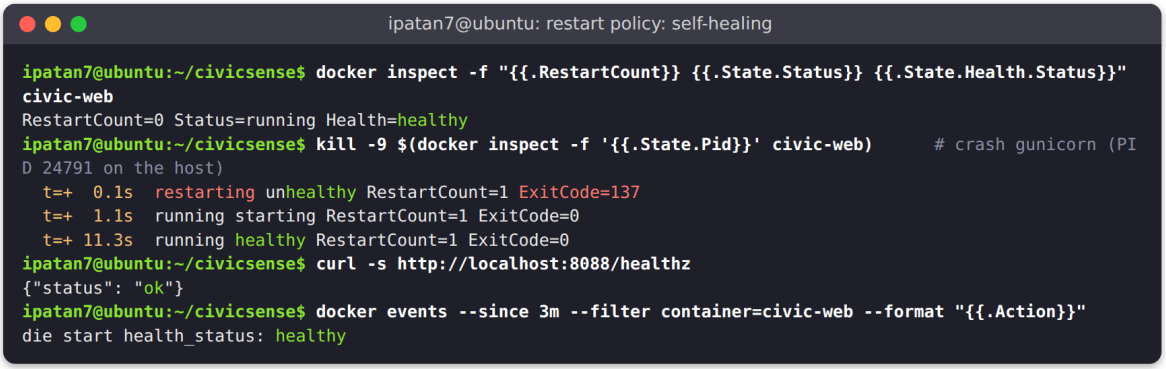


Figure 8.7 – Crash simulation under Compose: exit 137, automatic restart, healthy after 11.3 s

This test also exposed a defect. Because the entrypoint seeds demonstration data whenever `SEED_DEMO_DATA=1`, the automatic restart seeded a second time and the database grew from 120 to **240 complaints**. A restart must never change data. The fix is to treat seeding as a one-off task: it is now disabled for normal starts (`SEED_DEMO_DATA: "0"` in the Compose and stack files) and the demo data is loaded once with `manage.py seed_demo --reset`, as shown in Figure 8.9.

8.6 Docker Swarm: Replicas and Load Balancing

After `docker compose down` (which kept the volumes), a single-node swarm was initialised and the stack deployed. All services converged within seconds: three web replicas, two nginx replicas and one database replica (Figure 8.8). Because Swarm ignores `depends_on`, both nginx tasks started before the web service existed, failed with *host not found in upstream "web:8000"* and were replaced automatically by the swarm a few seconds later – self-healing at work even during deployment.

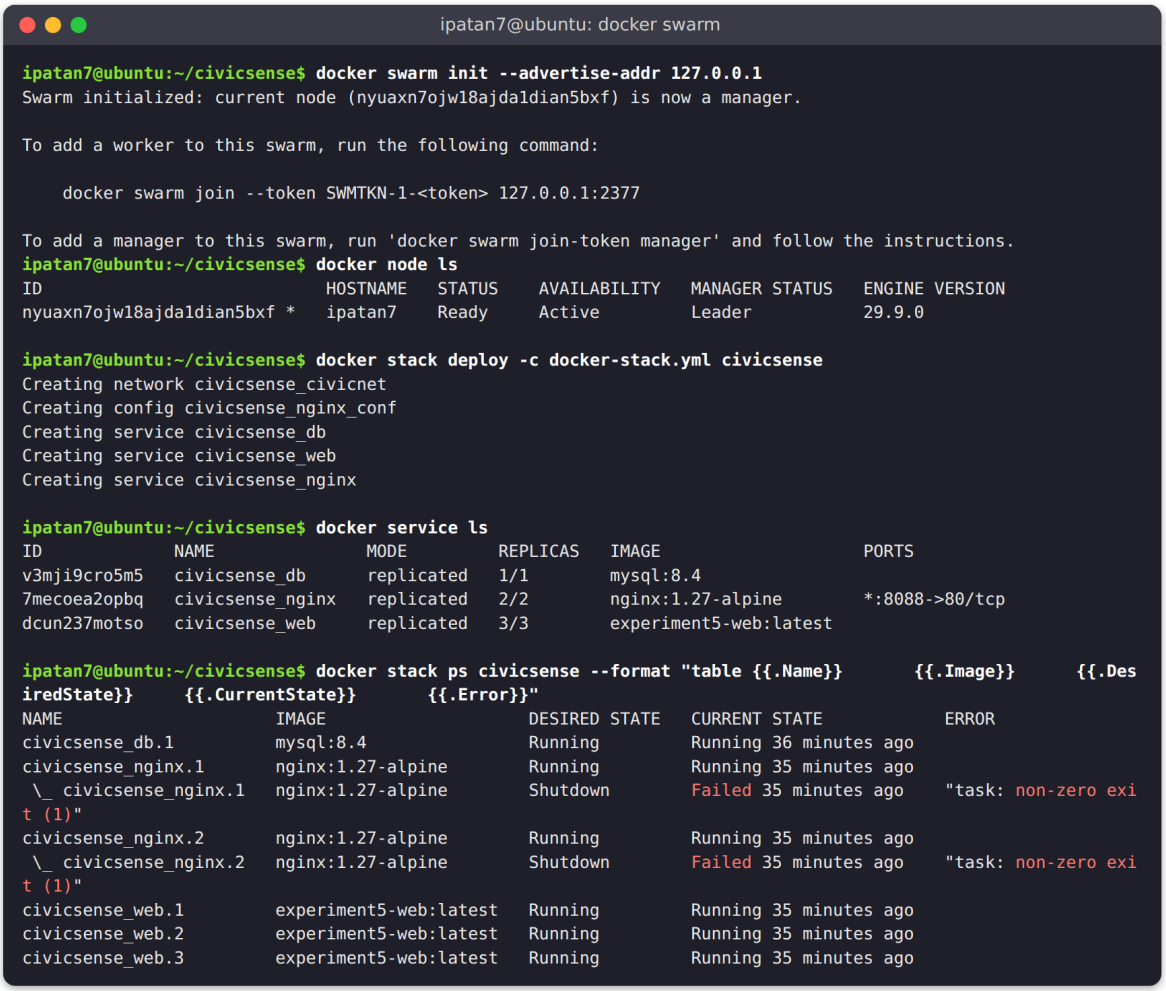


Figure 8.8 – Swarm initialised, stack deployed, services converged (nginx healed itself once)

The demonstration data was then reset with a one-off container joined to the attachable overlay network, the Docker equivalent of a Kubernetes Job. Inside the overlay network the name `web` resolves to a single virtual IP (10.0.1.5), while `tasks.web` lists the three replicas (10.0.1.6–10.0.1.8). Ninety requests sent to the published port all returned 200, and the gunicorn access logs show that the swarm's IPVS load balancer distributed them **exactly 30 / 30 / 30** across the replicas (Figure 8.9).

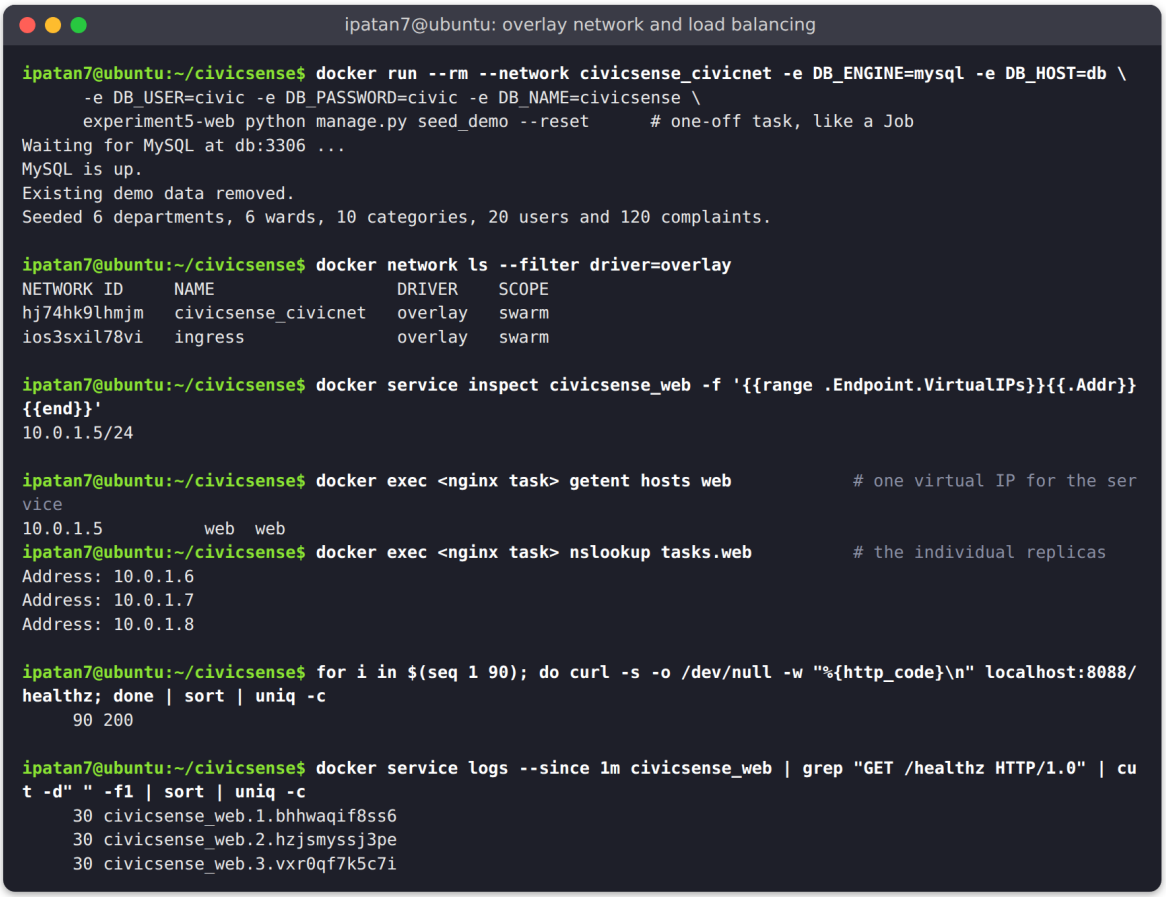


Figure 8.9 – One-off seeding task, service VIP and replica addresses, 90 requests split 30/30/30

8.7 Scaling

`docker service scale civicsense_web=5` added two replicas; all five were healthy **44 s** after the command, the time being dominated by each new replica's entrypoint and first health check. Scaling back to three removed two replicas immediately (Figure 8.10).

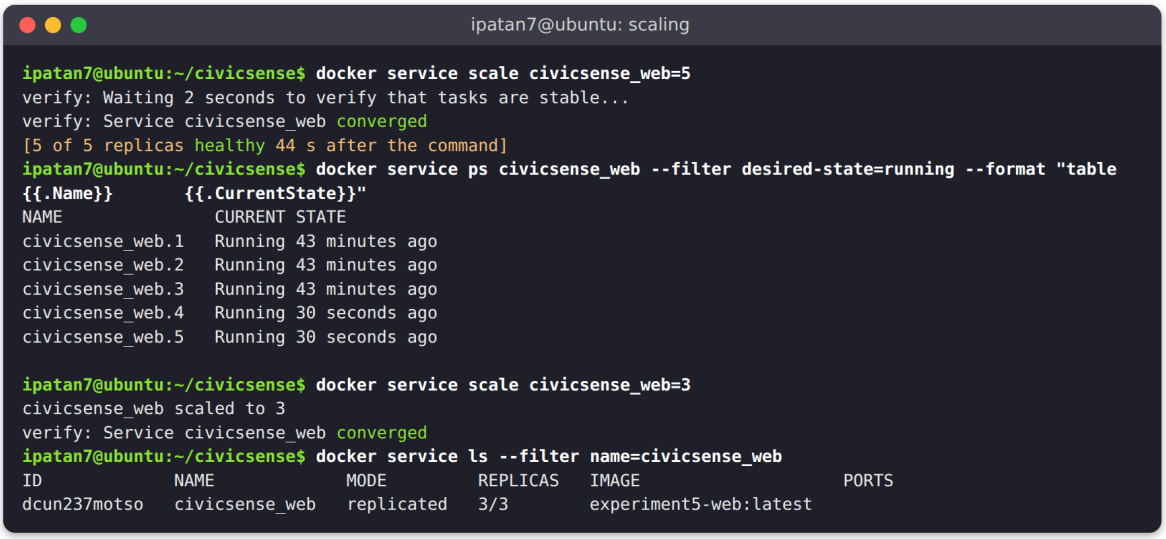


Figure 8.10 – Scaling the web service from 3 to 5 replicas and back to 3

8.8 Self-Healing in the Swarm

While a client sent a request roughly every 0.1 s for 60 s, one web replica was killed with SIGKILL. The service dropped to 2/3 replicas after one second, the swarm manager started a replacement task, and the service was back at 3/3 healthy replicas after **15.9 s**. During the whole minute **522 requests succeeded and none failed**, because the swarm stopped routing to the dead replica and the other two absorbed the load (Figure 8.11).

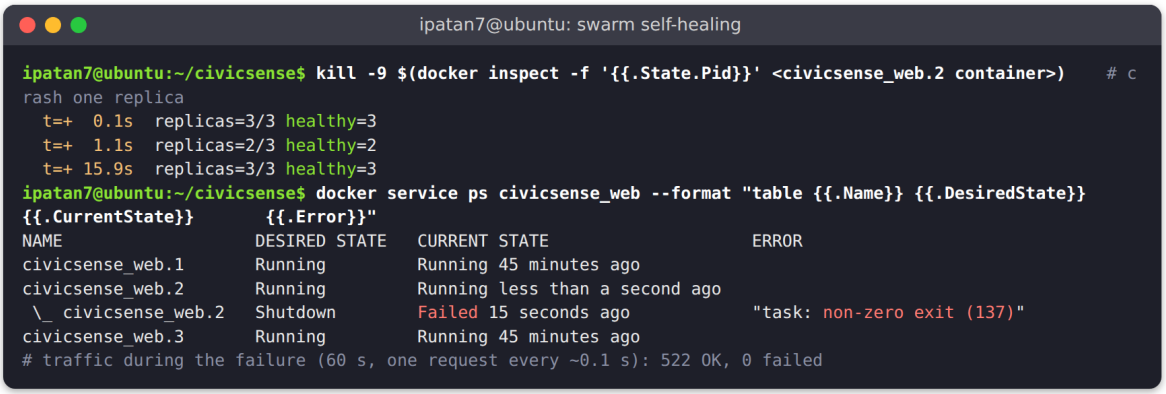


Figure 8.11 – Swarm self-healing: replica replaced, 522 requests OK and 0 failed

8.9 Rolling Update and Rollback

A new release was simulated by adding an environment variable to the service. With `parallelism: 1`, `delay: 10s` and `order: start-first`, the swarm started each new replica, waited until it was healthy, and only then stopped the corresponding old one. The update of three replicas took **94 s** and **840 requests were served with 0 failures**. `docker service rollback` then restored the previous version in **70 s** with **622 requests and 0 failures**, and the added variable was gone (Figure 8.12).

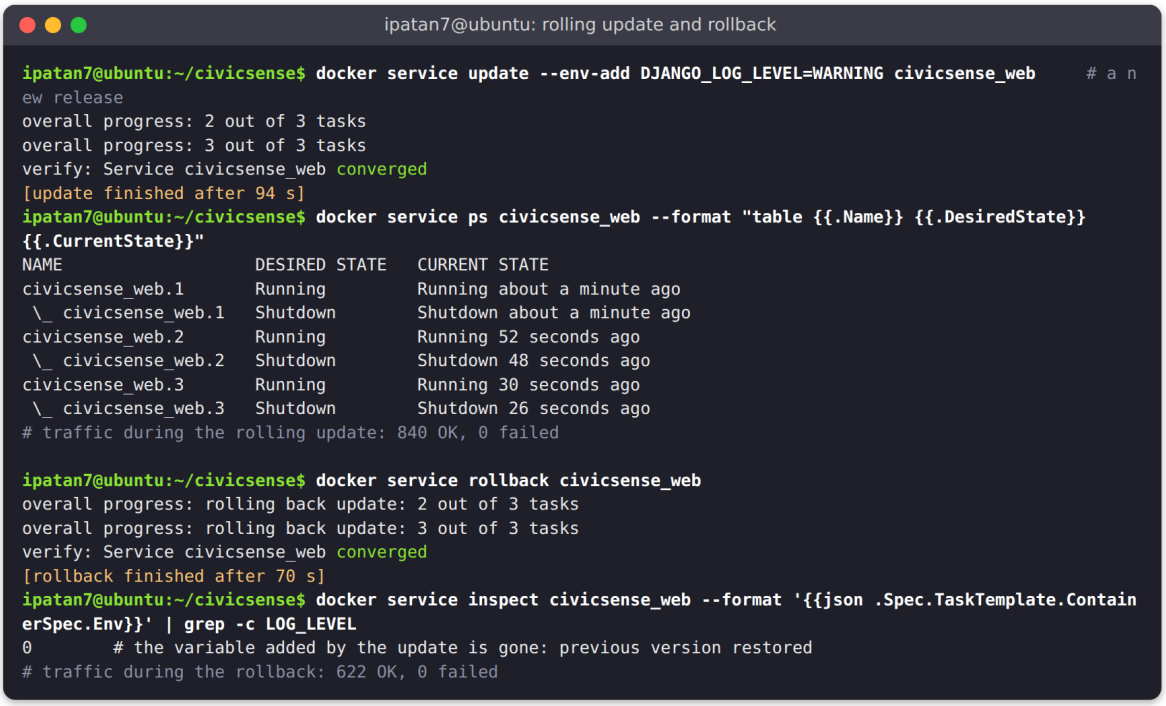


Figure 8.12 – Zero-downtime rolling update (840 OK, 0 failed) and rollback (622 OK, 0 failed)

8.10 Resource Usage

Table 8.2 – Measured resource usage (docker stats)

Container	Compose (no limits)	Swarm (limit 0.5 CPU, 512 MiB)
web (Django + gunicorn ×3)	150.2 MiB	121–122 MiB per replica
db (MySQL 8.4)	510.8 MiB	447.1 MiB
nginx	7.1 MiB	7.4 MiB per replica
Idle CPU	≈ 0.3 % in total	≈ 0.4 % in total

8.11 Summary of Results

Table 8.3 – Summary of measured results

Test	Result
Image build from scratch	187 s; 435 MB image (builder 698 MB)
Cold start to healthy (empty DB) / warm start	74 s / 18 s
Automated tests in the container on MySQL 8.4	39 / 39 passed in 85.8 s
Compose crash recovery	restarted in 1.1 s, healthy in 11.3 s
Swarm load balancing	90 / 90 OK, split 30 / 30 / 30
Scale 3 → 5 replicas	all healthy in 44 s
Swarm replica crash under traffic	healthy again in 15.9 s; 522 OK, 0 failed
Rolling update of 3 replicas	94 s; 840 OK, 0 failed

Test	Result
Rollback	70 s; 622 OK, 0 failed

8.12 Defects Found and Fixed

Running the application exactly as it is deployed revealed defects that never appear under `runserver` with SQLite. Table 8.4 lists them; the first two were found while building the stack in Experiment 5, the last three during the testing for this report.

Table 8.4 – Defects found and fixed

Defect	Symptom and cause	Fix
MySQL version	Django 6 refused to start: *NotSupportedError: MySQL 8.4 or later is required*; the stack used mysql:8.0	Compose file and CI service moved to mysql:8.4 LTS
Health probe through nginx	Container working but reported unhealthy: the `/healthz` location did not forward the Host header, so Django answered 400	`proxy_set_header Host $host;` added to that location
Re-seeding on restart	After an automatic restart the entrypoint seeded again: 120 → 240 complaints	Seeding disabled on normal starts; run once with `seed_demo --reset` as a one-off task
Login blocked by CSRF	Every login through nginx returned 403: *Origin checking failed – http://localhost:8088 does not match any trusted origins*; the trusted origins lacked the published port	Published port added to DJANGO_CSRF_TRUSTED_ORIGINS; login then returned 302 → dashboard 200
Start order in Swarm	nginx exited with *host not found in upstream* because Swarm ignores depends_on	Handled by the restart policy; nginx converges once the web VIP exists

CHAPTER 9: SCREENSHOTS

The screenshots in this chapter were taken from the containerised application. Figure 9.1 and Figure 9.2 were captured on 30 October 2026 from the Compose stack through nginx; the remaining figures show the same application pages captured during Experiment 5.

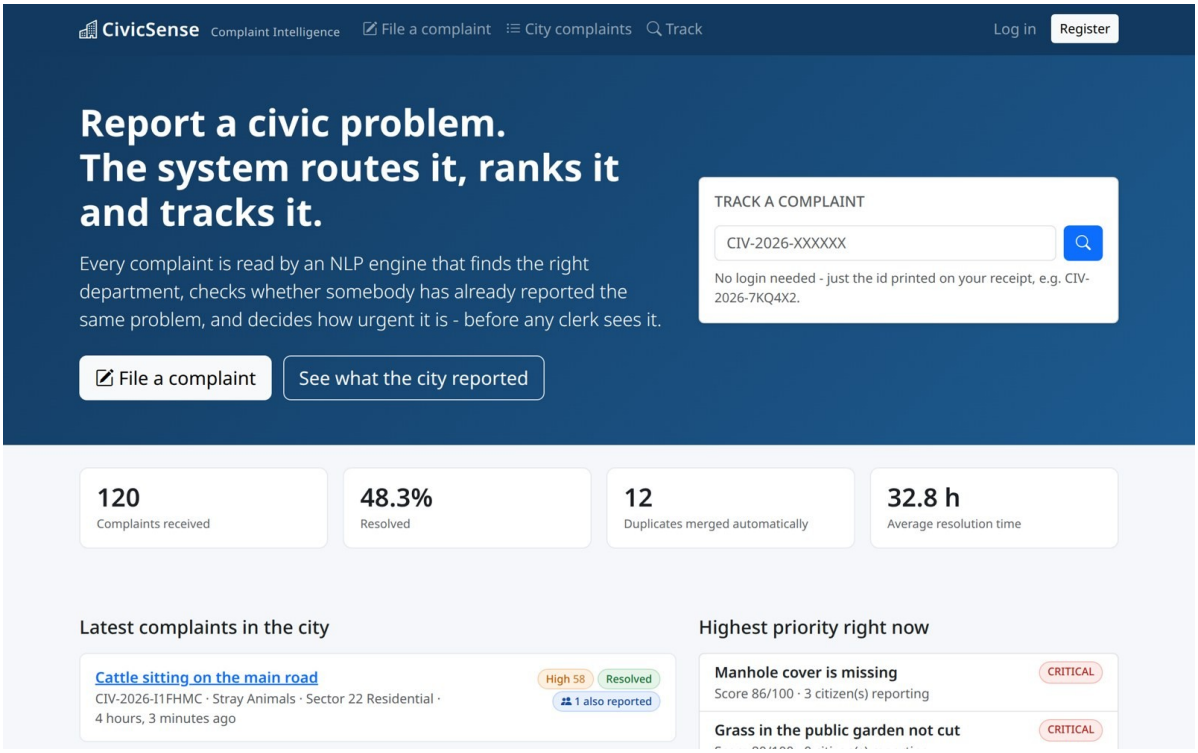


Figure 9.1 – Portal home page served by the containers through nginx (localhost:8088), live statistics read from MySQL

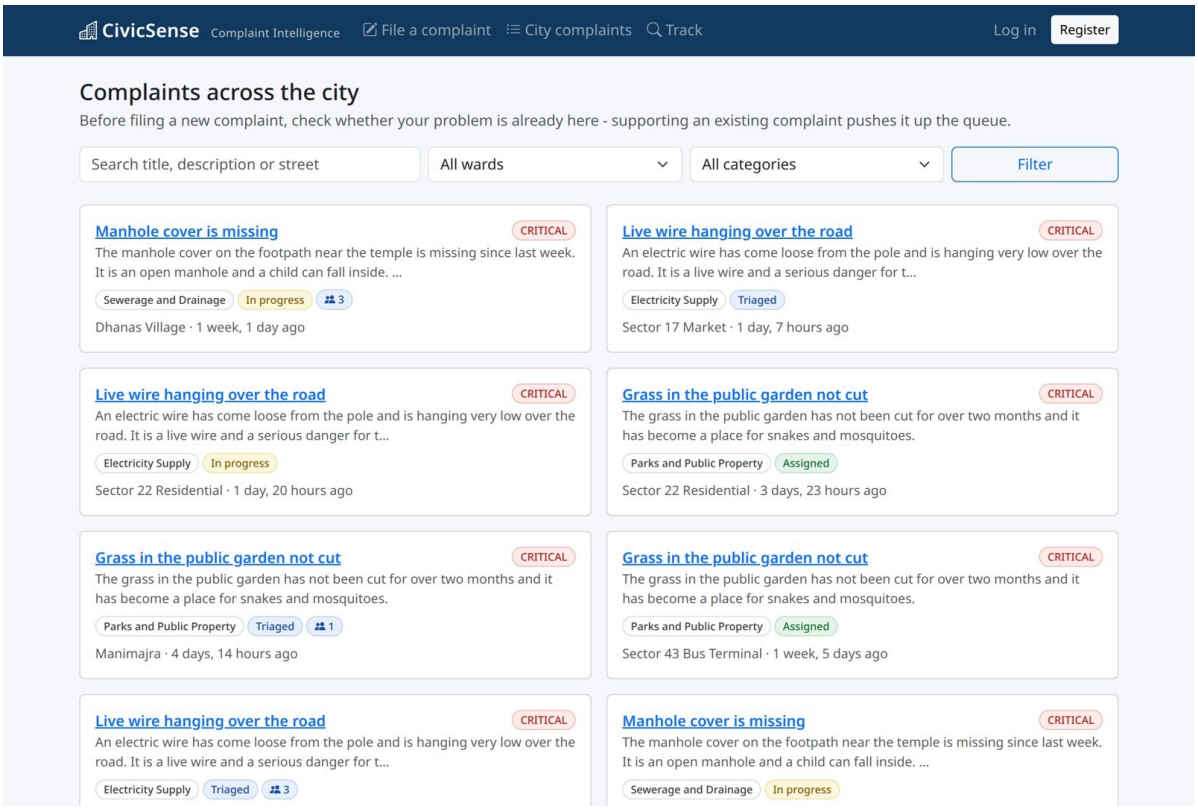


Figure 9.2 – City complaints page served from the Docker stack

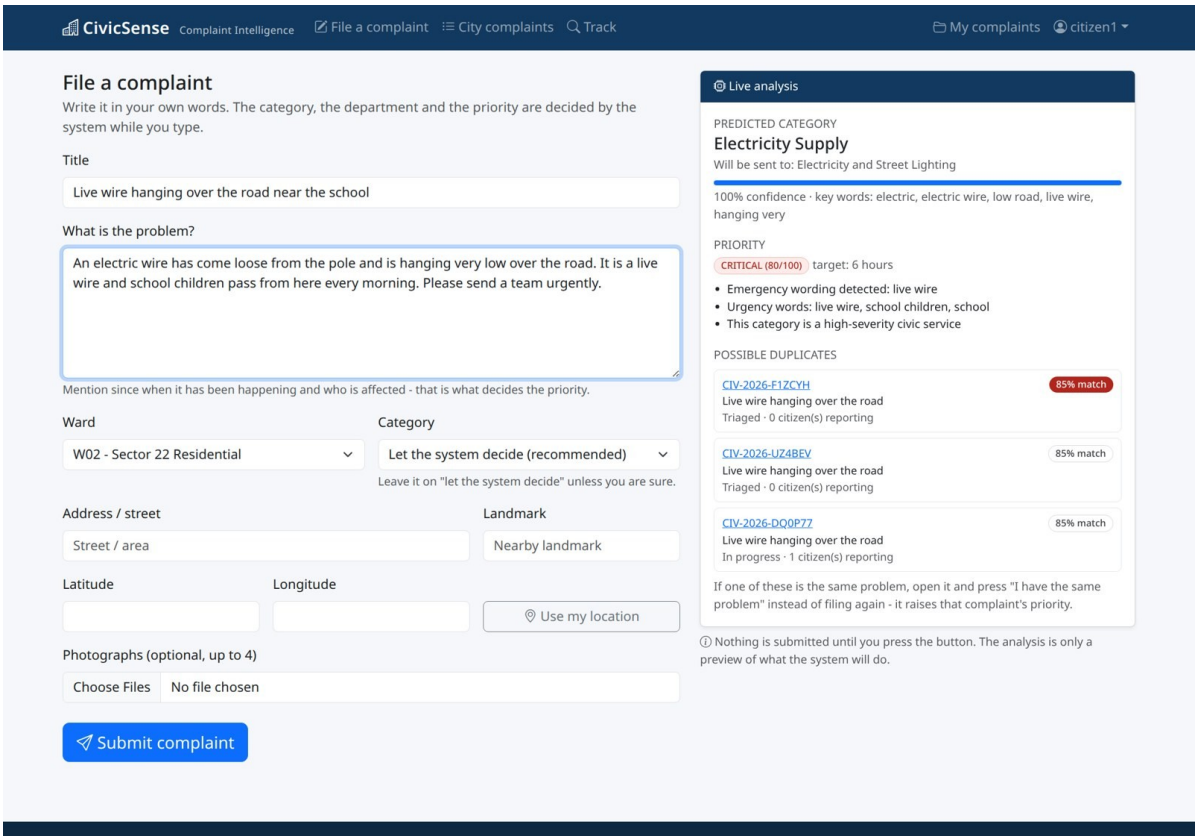


Figure 9.3 – Filing a complaint: category, deciding words and priority shown while the citizen types

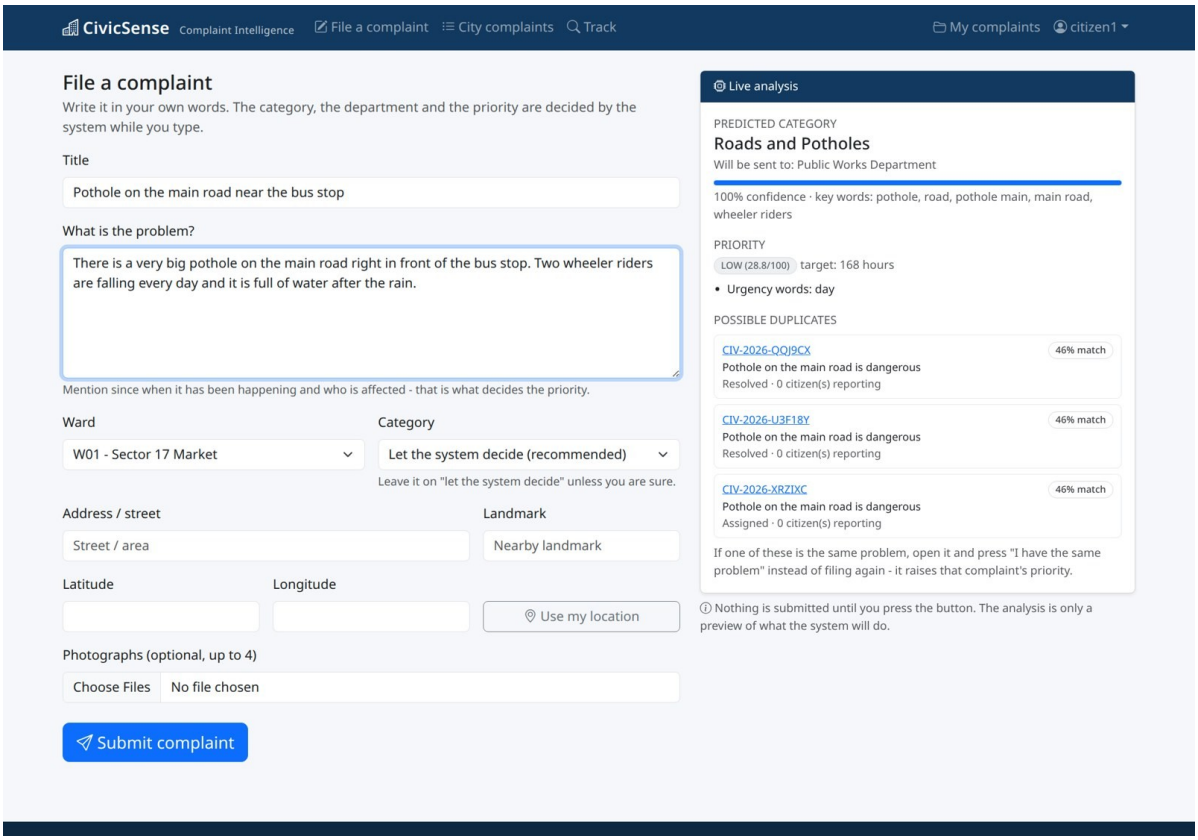


Figure 9.4 – Duplicate warning: a similar complaint nearby is found before submission

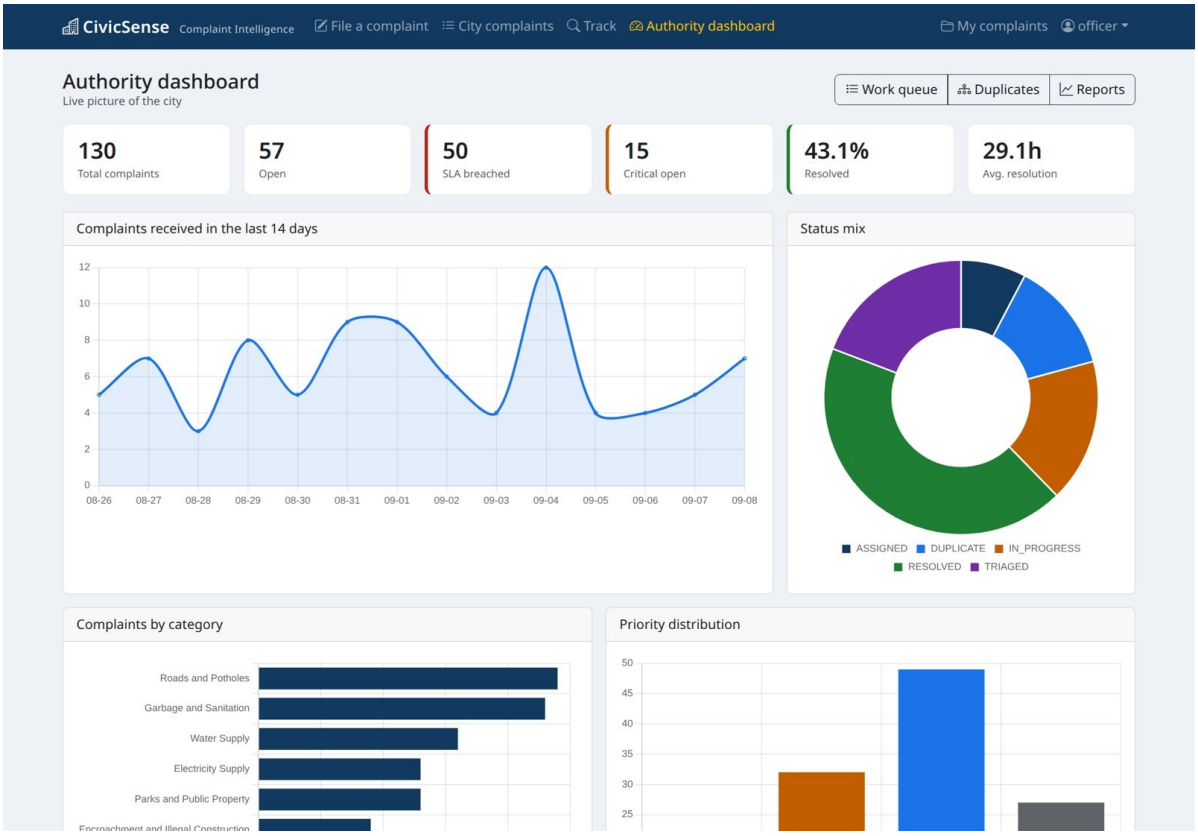


Figure 9.5 – Authority dashboard: KPIs, SLA breaches and ward hot-spots

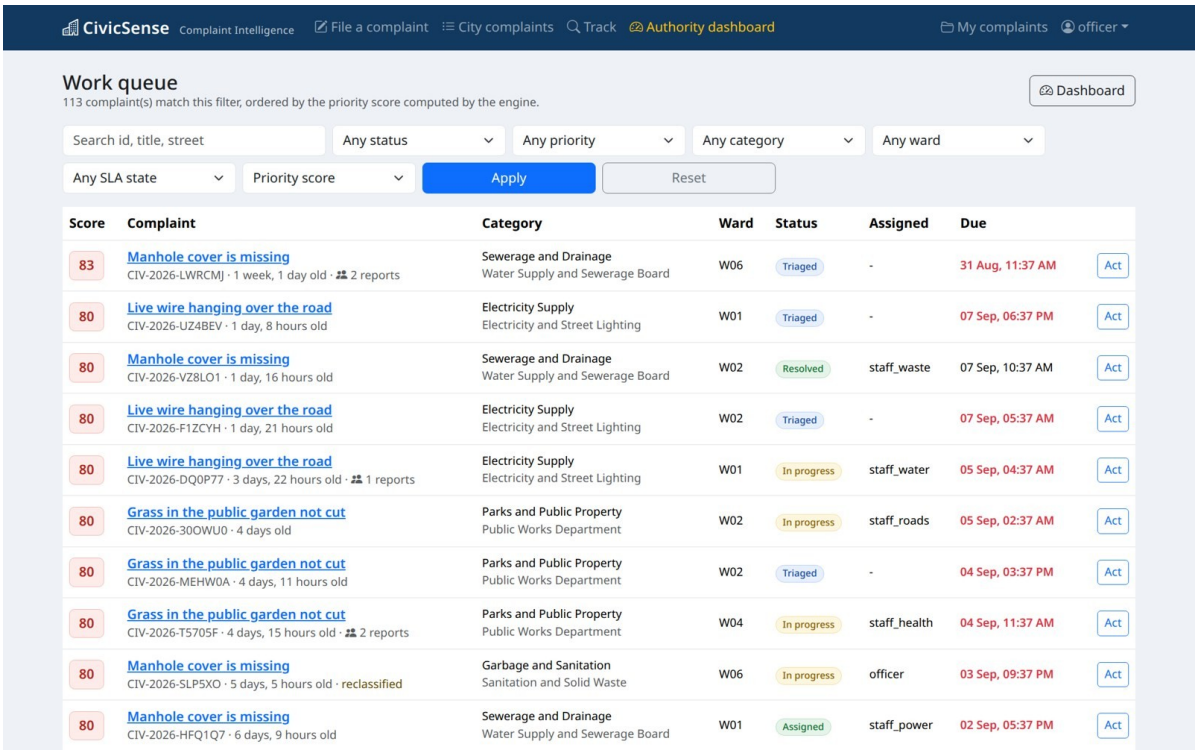


Figure 9.6 – Work queue ordered by the priority score with the SLA due date

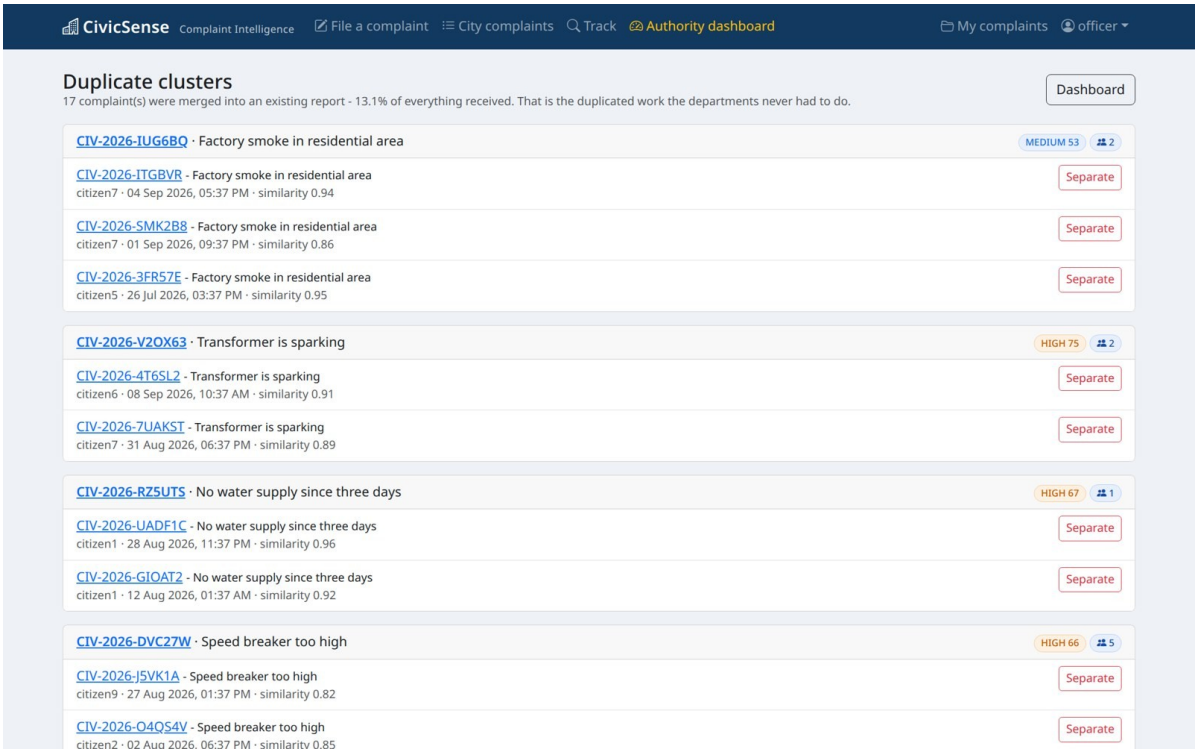


Figure 9.7 – Duplicate clusters merged by the engine

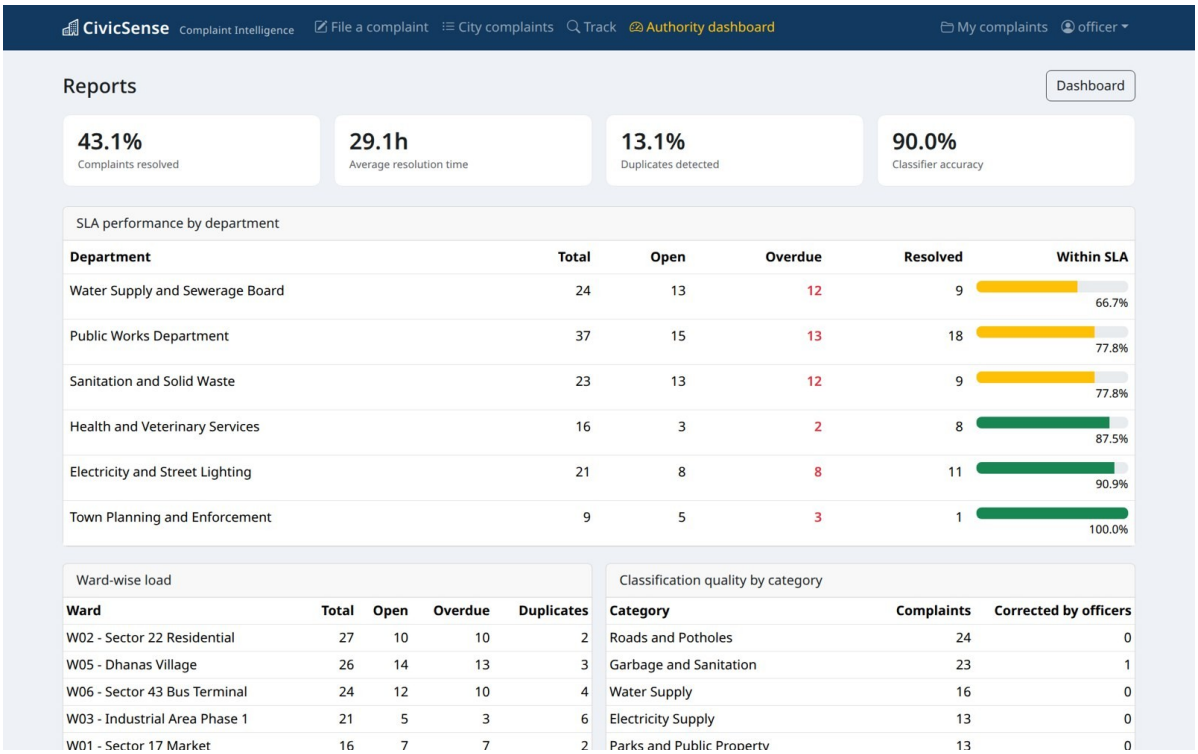


Figure 9.8 – Department-wise SLA report

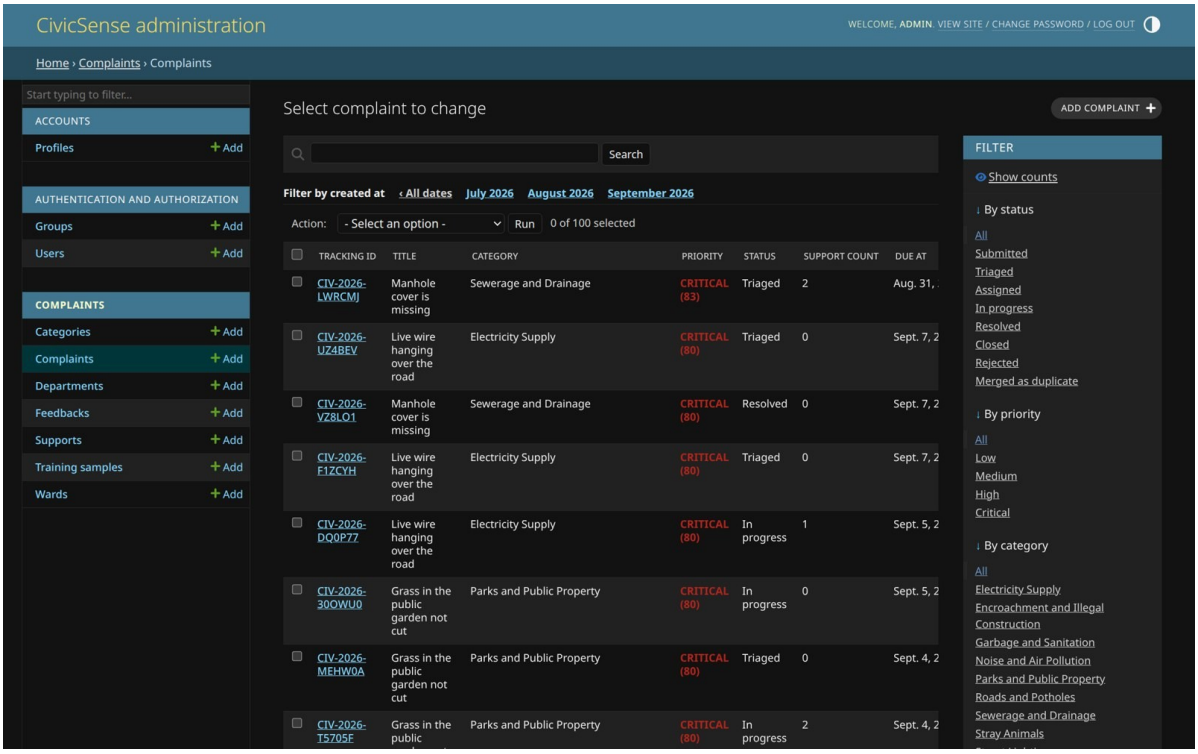


Figure 9.9 – Django administration of the containerised application

CHAPTER 10: CONCLUSION

This project took a complete, real application – CivicSense, a Django and MySQL platform with a custom natural-language engine – and made it portable, reproducible and resilient with Docker. A two-stage Dockerfile produces a 435 MB image that runs as an unprivileged user, contains no compiler and reports its own health. An entrypoint script turns the release procedure into code. One Compose file starts MySQL 8.4, the application under gunicorn and an nginx reverse proxy on a private network with persistent volumes, in the correct order, with one command.

The measurements show that the design works. All 39 automated tests pass inside the container against MySQL. A crashed container is restarted within about a second and is healthy again within about eleven. Under Docker Swarm the same image ran as three replicas behind a virtual IP that shared requests exactly evenly, scaled to five replicas in under a minute, replaced a killed replica in 15.9 s, and was updated and rolled back – in each case with **zero failed requests** out of nearly two thousand sent during these operations.

Just as important, running the application the way it is deployed exposed defects that a developer server could never show: a database version incompatibility, a proxy header missing from the health probe, data being seeded again on every automatic restart, and logins rejected because the trusted origins did not include the published port. Each was diagnosed from container logs and fixed in configuration, which is the practical meaning of the DevOps principle that the deployment environment must be tested rather than assumed.

Overall the project met its objectives: it demonstrated image building, layering and caching, multi-stage builds, container networking and DNS, volumes, health checks, restart policies, Compose orchestration, Swarm replicas, overlay networking, load balancing, scaling, self-healing, rolling updates and rollback, continuous integration and a path to Kubernetes – all on one real application.

CHAPTER 11: FUTURE SCOPE

- **Publish images to a registry** – extend the pipeline to push each commit-tagged image to Docker Hub or GitHub Container Registry, and scan it for vulnerabilities (Docker Scout or Trivy) before publishing.
- **Multi-node swarm or managed Kubernetes** – run the stack on three nodes so that a whole machine can fail, or apply the existing manifests to a managed cluster with the HPA enabled.
- **Managed or replicated database** – replace the single MySQL container with a managed database service or a replicated StatefulSet with scheduled backups.
- **Object storage for photographs** – move uploads from a shared volume to object storage, removing the last shared file system between replicas.
- **Secrets management** – move passwords from environment variables to Docker secrets or Kubernetes Secrets backed by a vault.
- **Observability** – add a Prometheus metrics endpoint, structured JSON logs collected by Loki and Grafana dashboards for request rate, errors and latency per replica.
- **HTTPS** – terminate TLS at nginx or at an ingress controller with automatically renewed certificates.
- **Smarter health checks** – a separate readiness endpoint that also checks the database connection, while keeping the liveness check independent of the database.

REFERENCES

[1] Docker Inc., "Docker Documentation – Dockerfile reference", https://docs.docker.com/reference/dockerfile/ (accessed October 2026).

[2] Docker Inc., "Multi-stage builds", https://docs.docker.com/build/building/multi-stage/.

[3] Docker Inc., "Compose file reference", https://docs.docker.com/reference/compose-file/.

[4] Docker Inc., "Swarm mode overview" and "Deploy a stack to a swarm", https://docs.docker.com/engine/swarm/.

[5] Docker Inc., "Start containers automatically (restart policies)", https://docs.docker.com/engine/containers/start-containers-automatically/.

[6] D. Merkel, "Docker: Lightweight Linux Containers for Consistent Development and Deployment", Linux Journal, no. 239, 2014.

[7] N. Poulton, Docker Deep Dive, Nielsen Book Services, 2023 edition.

[8] B. Burns, B. Grant, D. Oppenheimer, E. Brewer and J. Wilkes, "Borg, Omega, and Kubernetes", ACM Queue, vol. 14, no. 1, 2016.

[9] A. Wiggins, "The Twelve-Factor App", https://12factor.net/.

[10] Django Software Foundation, "Deployment checklist" and "How to deploy with WSGI", https://docs.djangoproject.com/en/6.1/howto/deployment/.

[11] Gunicorn, "Design and Deploying Gunicorn", https://docs.gunicorn.org/.

[12] NGINX, "ngx_http_upstream_module" and "ngx_http_proxy_module", https://nginx.org/en/docs/.

[13] Oracle, "MySQL 8.4 Reference Manual", https://dev.mysql.com/doc/refman/8.4/en/.

[14] The Kubernetes Authors, "Deployments", "Configure Liveness, Readiness and Startup Probes" and "Horizontal Pod Autoscaling", https://kubernetes.io/docs/.

[15] GitHub, "About service containers" and "Workflow syntax for GitHub Actions", https://docs.github.com/actions.

[16] M. I. Patan, "Experiment 5 – CivicSense: Containerisation, CI/CD and Cloud Deployment", Technical Training-II, Chandigarh University, 2026. Repository: https://github.com/iPatan7/TECHNICAL-TRAINING-II.

[17] M. I. Patan, "civicsense-docker: CivicSense containerised with Docker – source code, Docker files and project report", GitHub, 2026. https://github.com/iPatan7/civicsense-docker.

APPENDIX A: RUNNING AND DEMONSTRATING THE PROJECT

The steps below start the project on any machine with Docker and show every feature described in this report. They take about ten minutes.

A.1 Get the code and start the stack

```
git clone https://github.com/iPatan7/civicsense-docker.git
cd civicsense-docker
docker compose up -d --build          # first time: builds the image, about 3 minutes
docker compose ps                     # wait until web shows (healthy)
docker compose exec web python manage.py seed_demo --reset   # load the demo data once
curl http://localhost:8080/healthz    # {"status": "ok"}
# open http://localhost:8080
```

If port 8080 is already taken (on the author's machine it is used by Jenkins), start the stack with the override file from the repository, which publishes nginx on 8088 and lists that port in the trusted origins; the portal is then at http://localhost:8088:

```
docker compose -f docker-compose.yml -f docker-compose.demo.yml up -d --build
```

Table A.1 – Demonstration accounts (seeded data)

Username	Password	Role and landing page
admin	admin123	Administrator – dashboard and Django admin (/admin/)
officer	civic123	Ward officer – dashboard, assign, merge, reclassify
staff_roads	civic123	Department staff – only the roads department's queue
citizen1	civic123	Citizen – file and track complaints

A.2 Show the Docker features

```
docker compose logs web | head -30                      # entrypoint: wait, migrate,
gunicorn
docker inspect -f '{{.State.Health.Status}}' civic-web    # healthy
docker exec civic-web id                                  # uid=1000(civic): not root
docker exec civic-web getent hosts db                     # DNS by service name
docker image ls civicsense-web                            # about 435MB
docker history civicsense-web                             # the layers
docker stats --no-stream                                  # CPU and memory per container
docker exec -e DB_USER=root -e DB_PASSWORD=rootpass civic-web python manage.py test
```

A.3 Self-healing under Compose

```
sudo kill -9 $(docker inspect -f '{{.State.Pid}}' civic-web)   # crash the application
watch -n1 docker ps                                            # restarting -> up ->
(healthy)
```

A.4 Docker Swarm: replicas, scaling, self-healing, rolling update

```
docker compose down                                     # free the ports, keep the volumes
docker swarm init --advertise-addr 127.0.0.1
docker stack deploy -c docker-stack.yml civicsense
docker service ls                                       # web 3/3, nginx 2/2, db 1/1
docker service scale civicsense_web=5                   # scale out
docker service ps civicsense_web
docker service update --env-add DJANGO_LOG_LEVEL=WARNING civicsense_web   # rolling update
docker service rollback civicsense_web                  # roll back
docker stack rm civicsense && docker swarm leave --force   # clean up
```

A.5 Stop everything

```
docker compose down        # keeps the data volumes
docker compose down -v     # also deletes the data (fresh start next time)
```